



山西大学
SHANXI UNIVERSITY

2025 届硕士学位论文

基于 Zynq 的卷积神经网络加速器的研究 与设计

作者姓名	徐英俊
指导教师	王昆 副教授
学科专业	通信工程（含宽带网络、移动通信等）
研究方向	智能信息处理
培养单位	物理电子工程学院
学习年限	2022 年 9 月至 2025 年 6 月

二〇二五年六月

山西大学

2025 届硕士学位论文

基于 Zynq 的卷积神经网络加速器的研究 与设计

作者姓名	徐英俊
指导教师	王昆 副教授
学科专业	通信工程
研究方向	智能信息处理
培养单位	物理电子工程学院
学习年限	2022 年 9 月至 2025 年 6 月

二〇二五年六月

Thesis for Master' s degree, Shanxi University, 2025

**Research and Design of a Convolutional Neural
Network Accelerator Based on Zynq**

Student Name	Ying-jun Xu
Supervisor	A.Prof. Kun Wang
Major	Communication Engineering
Specialty	Intelligent Information Processing
Department	College of Physics and Electronic Engineering
Research Duration	2022.09-2025.06

June, 2025

摘 要

近年来,基于卷积神经网络(Convolutional Neural Networks, CNN)的目标检测算法因其快速的检测速度和高精确度,在自动驾驶、航空航天、质量检测等多个领域得到了广泛应用。然而,应用场景的不断扩展对硬件部署平台提出了新的要求。作为大多数目标检测算法主要运行平台的图形处理单元(Graphics Processing Unit, GPU),在功耗和成本方面面临显著限制。同时,通用的嵌入式平台往往难以满足神经网络对海量计算和参数的需求。

针对以上问题,本文选择在 Zynq-7020 平台上开展对目标检测模型的改进和算法的优化与设计,主要的工作内容包括:

(1) 从结构轻量化、参数定点化和硬件加速三个方面对 CNN 网络进行加速研究。首先在 LeNet-5 模型的基础上改进一种轻量化的 CNN 架构,通过 MNIST 数据集训练测试,识别准确率可达 98.2%。然后使用硬件描述语言 Verilog 把 CNN 架构部署在 Zynq 平台上,通过将参数定点量化为 int14 整数型以减少存储消耗,降低 CNN 架构的硬件部署难度,随后把卷积的并行性计算与流水线操作相结合,提高整体架构的实时性。仿真结果显示:在 50 MHz 时钟频率下,完成一次预测的时间为 86.02 μ s,与现有的轻量化架构相比,在实时性方面具有显著的优势。

(2) 从结构优化和硬件加速方面对 YOLOv4-tiny 网络加速研究,并应用到头盔目标检测中进行验证。先采用乒乓缓冲策略提高系统资源利用率,再通过模型稀疏化训练、通道剪枝以及数据量化的方法对 YOLOv4-tiny 网络进行优化,最后通过硬件平台的流水线特性对卷积和上采样模块进行加速。将改进后的目标检测模型部署在 Zynq-7020 平台,运行结果表明:在同等任务负载下,动态功耗仅为中央处理器(Central Processing Unit, CPU)平台的 2.5%,图像检测速度是双核 ARM-A9 平台的 8 倍,优势明显。资源消耗均衡,各种硬件资源消耗占比处于 30%-70%之间,避免因单一资源需求偏高而造成板卡成本过高的情况,解决了目标检测算法性能在边缘硬件平台中受资源限制的问题。

论文在 Zynq-7020 平台上分别实现了对 LeNet-5 和 YOLOv4-tiny 目标检测模型的加速和轻量化设计,在资源受限的边缘硬件平台上满足了目标检测算法对低成本、低功耗和高实时性的需求。

关键词：Zynq-7020；LeNet-5；YOLOv4-tiny；目标检测；硬件加速

ABSTRACT

Recently, Convolutional Neural Network (CNN)-based object detection algorithms have seen extensive use across multiple domains, including autonomous driving, aerospace, and quality inspection, owing to their rapid detection capabilities and high precision. However, the continuous expansion of application scenarios has imposed new demands on hardware deployment platforms. Graphics Processing Units (GPU), which serve as the primary running platforms for most object detection algorithms, face significant limitations in terms of power consumption and cost. Meanwhile, general-purpose embedded platforms often struggle to meet the extensive computational and parametric demands of neural networks.

To address the aforementioned issues, this paper focuses on improving the object detection model and optimizing/designing algorithms on the Zynq-7020 platform. The main work includes:

(1) Study CNN acceleration from three perspectives: lightweight architecture, parameter quantization, and hardware acceleration. Firstly, an improved lightweight CNN architecture was developed based on the LeNet-5 model. When trained and tested on the MNIST dataset, it achieved a recognition accuracy of 98.2%. The CNN architecture was then deployed on the Zynq platform using the hardware description language Verilog. To reduce storage consumption and lower the hardware implementation difficulty, the parameters were quantized into 14-bit fixed-point integers (int14). Additionally, parallel computing and pipeline operations were combined to enhance the real-time performance of the overall architecture. Simulation results showed that under a 50 MHz clock frequency, the time required for one prediction was $86.02\ \mu\text{s}$, demonstrating significant advantages in real-time performance compared to existing lightweight architectures.

(2) The YOLOv4-tiny network acceleration is studied from the aspects of structure optimization and hardware acceleration, and applied to helmet target detection for verification. First, a ping-pong buffering strategy is employed to enhance system resource utilization. The YOLOv4-tiny network is then optimized through sparse training, channel

pruning, and data quantization. Finally, the pipelining features of the hardware platform are utilized to accelerate the convolution and upsampling modules. After deploying the optimized object detection model on the Zynq-7020 platform, experimental results demonstrate significant improvements: Under the same workload, the dynamic power consumption is only 2.5% of that of a CPU platform, while the image detection speed reaches 8 times that of a dual-core ARM-A9 platform. Resource utilization remains balanced, with hardware resource consumption rates distributed between 30% and 70%, preventing excessive costs due to uneven resource demands. This approach effectively mitigates the performance limitations of object detection algorithms caused by resource constraints in edge computing hardware platforms.

The paper successfully implemented acceleration and lightweight design for both the LeNet-5 and YOLOv4-tiny object detection models on the Zynq-7020 platform, meeting the requirements for low cost, low power consumption, and high real-time performance of object detection algorithms on resource-constrained edge hardware platforms.

Keywords: Zynq-7020; LeNet-5; YOLOv4-tiny; Target Detection; Hardware Acceleration

目 录

1 绪论.....	1
1.1 课题研究背景及意义.....	1
1.2 国内外研究现状.....	2
1.3 论文的主要研究内容及章节安排	6
2 卷积神经网络相关技术	8
2.1 卷积神经网络基本结构.....	8
2.1.1 卷积层.....	8
2.1.2 激活函数层.....	11
2.1.3 下抽样层.....	14
2.1.4 全连接层.....	14
2.2 经典网络模型 LeNet-5	16
2.3 卷积神经网络加速方法.....	17
2.3.1 模型轻量化技术.....	17
2.3.2 硬件加速技术.....	18
2.3.3 软件框架优化技术.....	19
2.4 本章小结.....	20
3 基于 Zynq 的 CNN 目标识别算法设计与实现	21
3.1 CNN 架构轻量化设计	21
3.2 系统总体框架设计.....	23
3.3 模块加速设计.....	23
3.3.1 卷积模块加速设计	23
3.3.2 池化模块加速设计	24
3.3.3 卷积+池化模块的流水线设计	25
3.3.4 全连接模块设计	25
3.4 基于 Zynq 平台搭建卷积神经网络系统.....	26
3.4.1 Zynq 平台实现卷积神经网络方案分析	26
3.4.2 读写地址生成规律.....	27
3.4.3 存储方案选择及流水线设计.....	29
3.4.4 层间通信及计算协同.....	31
3.5 实验结果与分析.....	32
3.6 本章小结.....	34
4 基于 Zynq 的头盔检测技术硬件平台设计	35
4.1 YOLOv4-tiny	35
4.2 乒乓缓冲策略设计.....	37

4.3 算法的剪枝优化设计.....	38
4.3.1 模型稀疏化训练.....	38
4.3.2 通道剪枝.....	39
4.4 特征参数数据量化设计.....	42
4.5 高层次综合工具实现模块加速设计	44
4.5.1 卷积模块加速设计.....	44
4.5.2 上采样模块加速设计.....	45
4.6 实验结果分析.....	47
4.6.1 实验环境.....	47
4.6.2 平台搭建与上板验证.....	48
4.6.3 结果分析.....	50
4.7 本章小结.....	53
5 结论与展望.....	54
参考文献.....	56

1 绪论

1.1 课题研究背景及意义

人工神经网络（Artificial Neural Networks, ANNs）是一种受生物神经系统结构和功能启发的计算模型，旨在模拟人类大脑中神经元之间的信息处理方式^[1-3]。自 20 世纪 40 年代问世以来，神经网络经历了多个发展阶段，尤其是近年来随着深度学习的进步，使其成为机器学习和人工智能领域的关键技术之一^[4]。通过利用多层非线性变换，神经网络能够从数据中自动提取特征，并执行图像识别^[5,6]、自然语言处理^[7,8]和语音识别^[9,10]等复杂任务。尽管在计算资源和数据需求方面面临挑战，但其强大的建模能力和广泛的应用潜力使其成为现代人工智能研究的核心领域之一。神经网络的发展离不开早期对生物视觉系统的研究，以及后续在模式识别和深度学习领域的突破。

CNN 是一种专门用于处理具有网格结构数据（如图像）的深度学习模型^[11-13]。本质上，CNN 是一种深度前馈神经网络，其起源可追溯到人工神经网络中的感知机结构。自 Yann LeCun 于 1990 年提出 LeNet 以来^[14]，CNN 在图像分类、目标检测和语义分割等领域取得了显著成果。2012 年，AlexNet 在 ImageNet 竞赛中的突破性表现标志着 CNN 的复兴^[15]，并引发了深度学习的浪潮。LeNet-5^[16]作为最早的 CNN 模型之一，成功应用于手写数字识别，但其结构简单，仅适用于小规模任务。后来的 AlexNet、VGGNet 和 ResNet 等模型通过增加网络深度和引入残差连接等创新结构，提升了特征提取能力和性能。YOLO 系列^[17]通过将检测任务转化为回归问题，颠覆了传统目标检测方法，实现了端到端的高效检测。从 YOLOv1 到 YOLOv8，模型在速度和精度上不断优化，从学术工具演变为工业级产品，广泛应用于自动驾驶^[18]和安全监控^[19]等领域。这一过程体现了 CNN 从简单到复杂、从低效到高效、从学术到工业的全面演进。

在 CNN 的发展历程中，硬件条件的限制始终是制约其算力需求的关键因素之一。传统 CPU 虽然具备通用计算能力，但其运算性能的提升速度相对缓慢，难以满足深度学习模型^[20]对大规模并行计算和高吞吐量的苛刻需求。尤其是在处理高维图像数据或复杂网络结构时，CPU 的串行计算模式往往成为性能瓶颈，导致训练周期过长，严重限制了深度学习技术的实际应用场景。相比之下，图形处理器（Graphic Processing

Unit, GPU) 凭借其高度并行的计算架构和强大的浮点运算能力, 为深度学习提供了显著的加速效果^[21]。GPU 的数千个计算核心能够同时处理大量矩阵运算, 使其在训练深度神经网络时表现出远超 CPU 的效率, 成为当前主流的深度学习硬件平台。然而, GPU 的高性能也伴随着显著的局限性: 其功耗通常高达数百瓦, 需要复杂的散热系统支持, 这不仅增加了硬件部署成本, 还导致能耗居高不下。此外, 高端 GPU 的采购成本昂贵, 且需要专门的软件框架支持, 进一步提高了技术门槛。这些因素使得 GPU 在移动设备、嵌入式系统或其他对功耗敏感的低功耗应用场景中适用性大幅降低。因此, 如何在有限功耗条件下实现高效的深度学习计算, 成为当前学术界和工业界亟待解决的核心问题之一。

过去十年中, 专用集成电路 (Application Specific Integrated Circuit, ASIC) 和现场可编程门阵列 (Field Programmable Gate Array, FPGA) 的性能稳步提升, 并在图像识别^[22]、密集型运算^[23]等领域应用广泛。FPGA 的并行计算能力、高灵活性、低功耗等优势, 使其在多个领域中具有重要的应用价值。随着半导体技术的不断进步, ASIC 和 FPGA 在未来将继续发挥重要作用, 推动硬件加速技术的发展。

嵌入式设备的计算资源、存储空间和功耗有限, 难以直接运行复杂的 CNN 模型。许多嵌入式应用还会对计算速度和响应时间有严格的要求, 复杂的 CNN 模型难以满足这些需求。如何降低 CNN 的计算复杂度和资源需求, 同时保持较高的性能^[24], 如何在资源受限的嵌入式平台上实现对 CNN 图像分类算法的加速, 在移动设备上降低成本和功耗的等题, 日益成为深度学习领域的研究热点之一。

1.2 国内外研究现状

LeNet-5 是计算机视觉领域的里程碑式模型, 被成功应用于银行支票的手写数字识别, 证明了深度学习在实际任务中的潜力^[25]。LeNet-5 在手写数字识别任务中实现了 98% 的准确率, 这一结果远超支持向量机、K 近邻算法等传统的机器学习方法。与其它数字识别方法相比, LeNet-5 能够以最少的预处理实现最好的识别效果, 这表明 CNN 具有强大的特征提取能力。LeNet-5 的成功使 CNN 在学术界引起了广泛关注, 为后续 AlexNet、VGG 和 ResNet 等深度学习模型的发展奠定了基础。随着 GPU 的发展, CNN 的计算能力得到了显著提升。GPU 具有强大的并行计算能力, 能够高效处理卷积运算, 从而解决了早期 CNN 面临的算力不足问题, 并能够训练更深、更复杂的模型^[26]。随着 CNN 的发展, 模型的规模和计算量不断增加, 这使得模型只能在

特定平台上运行，而无法移植到资源受限的小型硬件平台。

为了解决这一问题，学术界开始了 CNN 模型小型化和轻量化的研究。Pitonak 等人探索了量化过程对 CNN 架构的影响，调整硬件架构设计空间以确定最佳的 FPGA 资源利用率，其量化权重和激活对模型性能的影响较小，最终，内存占用的减少使得模型能够部署在低成本的 FPGA Xilinx Zynq-7020 上^[27]。Chun 等人先后把高精度的浮点型数据量化成 8 位与 4 位的低精度的定点型数据，并对模型进行训练，只产生了微小的精度损失^[28]。该文献在 FPGA 平台上搭建 LeNet-5 模型，并对数据进行 4 位定点量化，结果显示，此模型占用了大量的 DSP 资源，提高了开发板在成本上的要求。Rama 等人提出了一种可重用的量化硬件架构来加速深度 CNN 模型。在 CNN 模型中，卷积的计算使用了 25 个处理元素^[29]。流水线、循环展开和数组划分是提高卷积层和全连接层计算速度的技术。在低成本、低内存的 Xilinx Pynq-Z2 系统芯片边缘器件上进行了 MNIST 手写数字图像分类测试，准确率在 98% 以上。由于 Pynq-Z 板上有 Python 接口，许多研究会首选这种开发板做 CNN 加速研究，并利用 Python 软件进行精确的权值和偏置训练。因此，Python 在 PS（Processing System，处理系统）中运行以控制 PL（Programmable Logic，可编程逻辑）中的覆盖设计，使得 FPGA 更易集成于计算机视觉和机器学习应用中。但是，也意味着不带有 Python 接口的 FPGA 开发板无法使用这种 CNN 模型，不具有普遍性。

卷积涉及到一个二维的乘加计算（Multiply Accumulate，MAC）^[30]。在数学形式中，它可以被看作是由几个乘-累加运算组成的。 3×3 卷积结构充分利用卷积运算的并行性，通过设置行缓存结构，实现了图像的卷积窗口滑动，形成了高效并行流水线操作的卷积计算电路。输入的特征图数据填满整个管道后，每个时钟周期相当于完成一个 3×3 的卷积窗口滑动，输出一个卷积结果。它相当于在一个时钟周期内完成 9 次乘法运算和 8 次加法运算，与串行计算处理器相比，速度提高了 17 倍。卷积核的尺寸越大，加速效果越明显。但是这种结构只针对固定大小的卷积，难以适应实际 CNN 中多个卷积层的不同卷积方式，灵活性不够。Fei 等人提出一种基于 FPGA 的资源复用架构的 CNN 处理器设计理念，将 LeNet-5 模型中的卷积核全部统一为 3×3 大小的卷积核，降低硬件资源消耗和功耗统一的同时，实现对 CNN 算法的加速效应^[31]。LeNet-5 模型中的 5×5 卷积核被统一为 3×3 卷积核后，对手写体数字识别的准确率有所下降。Rui 等人采用可重构的卷积模块进行结构设计，通过改变参数可以动态调整成不同的卷积模块，该可重构结构在使用过程中得到了 11×11 的卷积输

出,在下一级的池化操作中,只池化了 10×10 的信息,信息损失了 17.4%,这也是导致手写体数字识别的准确率有所下降的原因之一^[32]。Wang 等人用 SqueezeNet 的 Fire 模块来替换 OctConv 中的传统卷积层,从而形成一种新的双重神经架构: OctSqueezeNet,同时提高了网络的准确性和效率^[33]。模型在保证 CNN 性能的基础上,通过降低卷积核尺寸、通道剪枝、降低 CNN 模型深度等手段,减少 CNN 模型的参数与计算量同样可以达到轻量化目的。

Zhao 等人提出了一种多 CNN 任务实时切换方法,在 FPGA 上实现卷积任务的并行运算方式,展现了 FPGA 平台的优势^[34]。利用嵌入式平台完成对 CNN 模型的加速设计,主要针对功耗、成本以及实时性等方面进行加速。April 等人提出一种用于资源有限设备的轻量级神经网络的可扩展设计体系结构^[35]。介绍了几种流水线和并行结构的 FPGA 在手写体数字分类中的应用。Jian 等人提出了一种基于峰值时间相关可塑性反向传播(BP-STDP)算法的多层 SNN 在线训练加速器,训练模块重用隐藏层神经元模块中的集成和火算法单元电路,大大降低了硬件开销^[36]。在能量有限的嵌入式系统上实现的脉冲神经网络对本研究具有很大的借鉴意义。

基于 FPGA 加速的对 CNN 的研究在许多与 AI 相关的领域早已展开,如 Google 的 TPU 在性能方面的优势以及在训练问题上的探索^[37],MIT 的 Eyeriss 可能是在计算机体系结构、深度学习硬件加速等领域的研究成果^[38],中科院计算所下的寒武纪在 AI 芯片领域对芯片架构及指令集的设计等^[39]。CNN 模型实现分为训练和推理两部分,FPGA 因其高性能、低功耗和可重构性等优势,已成为 CNN 硬件加速的一个有前景的研究平台。在设计基于 FPGA 的 CNN 硬件加速器时,需要综合考虑多个关键因素,如:卷积加速单元的硬件结构、存储及数据传输的特性和功耗与数据流动的关系等,以确保加速器在性能、功耗和资源利用率之间达到最佳平衡^[40]。Dan 等人提出了一种基于 FPGA 的柔性结构 CNN 加速器,以实现 MNIST 手写数字字符的识别^[41]。系统采用深度流水线处理,从粗粒度和细粒度两个层次对层间和层内并行性进行优化。实验中使用了 571 块 DSP 与 596 块 RAM 资源。这两项实验都只能在高端边缘设备上实现,在设备的选择上具有一定的局限性。Tianling 等人基于 LeNet-2 和 LeNet-3 两种模型研究 CNN 训练与推理集成的高算力实现方法,提出了采用以 FPGA 为核心的高性能异构架构(HA)器件实现 CNN 训练与推理过程^[42]。然而,这种方法目前也有一些局限性,并且对于广泛部署来说成本较高。Yangyang 等人以 LeNet-5 网络结构为基础,在多核异构架构器件 MPSoC 上使用多核 CPU 进

行网络训练的过程，实现人工智能训练和推理的集成，展现了高性能异构架构器件的优势^[43]。Akshay 等人采用 ARM 处理器+FPGA 芯片的形式在同一个硬件设备上完成了参数的训练部分和 CNN 实现部分^[44]。但其训练部分所达到的准确率只有 80%，效果较差。Li 等人利用现场可编程门阵列硬件平台，以 Verilog 代码实现 BP 算法，利用 Quartus13.0 和 modelsim 仿真与分析，为本次实验提供了理论支持^[45]。使用 HLS（High-Level Synthesis，高层次综合）工具的开发周期短，适合前期功能、性能的验证，可以节省时间和人力成本。但是，因为统一的接口和标准化的翻译，会产生一定的冗余资源，且在 HLS 代码生成 RTL 过程中，会引入工具 bug 或者限制，HLS 对资源和时序的控制能力较低。做为一名 Verilog 语言的学习者，使用 RTL 代码更有利于发挥 FPGA 的优势。

Li 等人提出了一种基于 FPGA MPSoC 方法的低光目标检测低阶图像增强技术，基于低光训练集训练了 YOLOv3 目标检测模型，并在 MPSoC 板上实现了边缘目标检测系统的高检测效率^[46]。但是，FPGA 资源利用率仍有优化空间，部分硬件资源未充分利用，且对于小目标检测任务，精度略有下降，不具有普遍性。

Nguyen 等人基于 FPGA 架构设计了一个 YOLOv4 模型，用以解决对飞行物的检测，主要解决的问题是浮点资源有限与实时性要求较高的问题，通过格式转换和定点量化等技术完成了对目标模型的加速^[47]。但是，该设计在功耗、实时性等方面的加速效果不显著，应用价值偏低，且量化过程引入了轻微的精度损失，可能影响复杂场景下的检测效果，硬件设计复杂度较高，开发周期较长。

Reis 等人针对 YOLOv3-tiny 模型提出了一种多引擎架构和设计流程，通过定点化策略和并行化设计，在 FPGA 平台上实现了高效的目标检测^[48]。实验把高精度的数据量化成低精度的 4 位定点数据，虽然通过这种方法降低了开发板的成本，但是，这种方法引入了较多的精度损失。

Zhang 等人设计了特征图填充和传输的集成自动化，显著减少了 DRAM 访问，并提高了在 DRAM 和片上 SRAM 之间传输特征图的速度，并通过 YOLOv4-tiny 模型的卷积层进行了测试，加速效果明显^[49]。然而，这种设计不适用于 YOLOv4-tiny 模型的整体架构，随着层数的加深，DRAM 存储和读取数据的弊端会扩大，增加整体设计的复杂性，增加控制端的时间开销，同时提高了 PFPGA 硬件实现的成本，使整体设计不具备应用普遍性。

Pan 等人利用 PFPGA 平台设计了一种低成本、便携式的成像设备，通过改进

YOLOv4-tiny 模型的电路设置、定点量化以及模型压缩等技术,减少了计算复杂度和存储需求,利用 HLS 工具完成全流程加速,在边缘设备上实现了高能效比,适合资源受限的嵌入式平台^[50]。但是,该设计仅适用于单一鱼类的目标检测,且 FPGA 资源利用率仍有优化空间,部分硬件资源未充分利用。

用低精度数据替代高精度浮点数据可以在不明显牺牲分类精度的情况下,显著提升训练和推理性能以及资源利用率^[51]。CNN 剪枝技术通过移除不重要的参数或神经元,减少模型的规模和计算量,同时还可以保持较高的精度^[52]。剪枝可以将数据规模减少十倍以上,而精度损失几乎可以忽略,这使得剪枝技术在资源受限的设备中具有重要的应用价值^[53]。

通过以上的对比,了解了国内外的研究现状,采用 FPGA 设备来实现 CNN 算法,利用高度并行的硬件结构可以显著加速推理过程。动态重配置特性可适配不同 CNN 模型或任务需求。合理简化 CNN 模型,可以提高该模型的资源复用率和结构的并行度;数据从浮点型向定点型的转化可以减少硬件资源的消耗,用逻辑资源实现乘运算,减少 DSP 模块的使用,充分发挥 FPGA 在成本上面的优势。提高 FPGA 开发板上各种资源的利用率,同样能够降低 FPGA 的成本。把 CNN 模型的训练和推理两部分集中到同一块开发板上,对 CNN 的跨平台应用也就具有了更加重要的意义。

1.3 论文的主要研究内容及章节安排

通过对 CNN 及其在嵌入式平台上应用的研究现状进行分析,本文把 CNN 轻量化和硬件加速技术作为研究重点,并且在 Zynq 平台上完成 CNN 加速器的设计。论文分为五个部分,各个章节内容如下:

第一章,简述了论文的研究背景和意义,介绍了 CNN 轻量化以及硬件加速技术的研究现状,并对论文各个章节进行规划和简述。

第二章,首先分析了 CNN 的基础拓扑结构和内部计算模块,其次对 LeNet-5 经典模型的架构进行详细分析,最后对模型剪枝、量化和知识蒸馏等轻量化技术进行简述。

第三章,在 LeNet-5 的基础上进行模型轻量化设计,首先通过降低卷积核尺寸来减少参数量,其次通过增加网络深度来补偿模型精度的损失,然后通过研究最佳的定点量化范围来降低卷积层和全连接层对模型精度的影响,最后利用 Zynq 的并行性与可重构性来实现对 CNN 加速的研究与设计。

第四章，首先分析了 YOLOv4-tiny 模型的基本结构，其次通过乒乓缓冲、通道剪枝和特征值量化技术完成 YOLOv4-tiny 模型的轻量化设计，然后利用 HLS 工具实现对卷积模块和采样模块的加速器设计，最后在 Zynq-XC7Z020 上完成模型搭建与板级验证。

第五章，对本文研究内容以及创新点进行总结与介绍，对未来的发展方向进行展望。

2 卷积神经网络相关技术

本章的主要内容是围绕 CNN 的相关技术展开讨论，首先阐述 CNN 的基础拓扑结构和内部计算模块，然后以 LeNet-5 为实例，详细分析 CNN 的层间结构组成，最后分析如何运用剪枝、量化、知识蒸馏等技术对 CNN 模型进行轻量化设计。

2.1 卷积神经网络基本结构

如图 2-1 所示，CNN 的基础结构主要由卷积层、池化层、全连接层和激活函数等核心组件构成^[54]，这些组件协同工作，通过层次化的方式从输入数据中提取特征，并最终完成特定的任务。

卷积层主要功能是捕捉输入数据（如图像）中的边缘、纹理等低级特征，以及更复杂的高级特征，是 CNN 的核心组件。池化层主要功能是对卷积层生成的特征图进行下采样，降低计算复杂度，防止过拟合。全连接层主要功能是特征整合，并映射到最终的输出空间。激活函数主要功能是为网络引入非线性特性，也是 CNN 中不可或缺的组成部分。

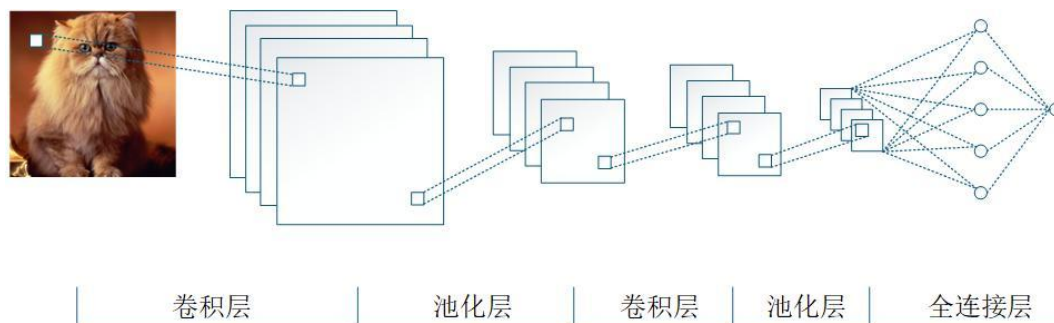


图 2-1 CNN 基础结构

Fig.2-1 The basic structure of CNN

2.1.1 卷积层

卷积操作通过滑动卷积核（或滤波器）的窗口在输入数据上进行计算，每个卷积核可以看作是一个特征检测器，用于捕捉输入数据中的目标特征^[55]。卷积操作的核心是卷积核与输入数据的局部区域进行逐元素相乘并求和，具体来说，卷积核是一个小矩阵，它会在输入数据上滑动。每次滑动时，卷积核与输入数据的对应局部

区域（进行逐元素相乘，然后将所有乘积结果相加，得到一个输出值。卷积操作的计算公式如（2-1）所示：

$$\text{conv}(IN, W, B) = B + \sum_{i=0}^{\text{KernelSize}} IN_i \times W_i \quad (2-1)$$

其中， IN 表示输入特征值， W 表示权重参数， B 表示偏置参数。

图 2-2 所示，卷积层的输入特征图的尺寸设置为 9×9 ，单一通道的卷积核大小为 3×3 （偏置参数为 0），步幅为 1，卷积层的输出特征图的尺寸为 7×7 。按照公式（2-1）计算可以得出，滑动窗口（输入的图片中的 3×3 黑色框）的 9 个特征值与 3×3 卷积核（红色的 3×3 红色框）的 9 个权重参数先按顺序进行乘运算，再把乘运算的结果相加，最后，得到的值对应的是输出特征图的第一个特征值（输出值中左上角 1×1 加深的红色框）。这一次卷积运算完成后，滑动窗口会向右移动一个单位，然后，再按照上述操作进行第二次卷积计算，得到输出特征图的第一行第二列对应的特征值。当滑动窗口完成第一行的卷积计算后，会先向下移动一个单位，再从左向右依次进行卷积操作，直到计算完成 7×7 输出特征图的全部特征值。

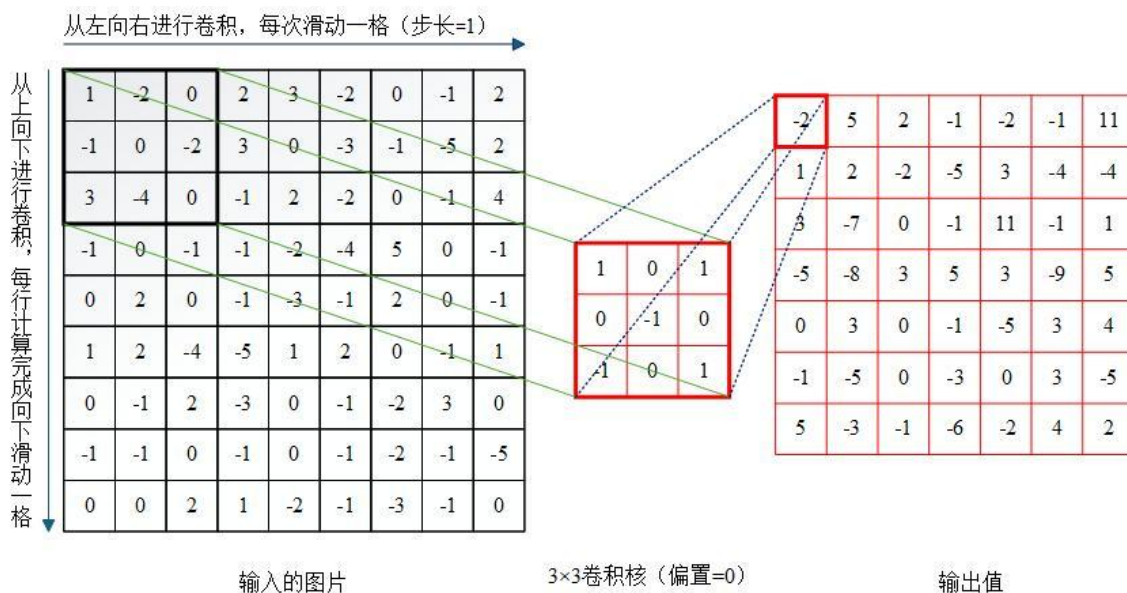


图 2-2 单通道卷积运算

Fig.2-2 Single-channel convolution operation

卷积层都是通过多个通道的卷积计算来提取输入特征图的所有特征， N 通道的卷积层意味着要使用 N 个不同的卷积核，其计算方式相对于单通道卷积层的卷积计算更为复杂，以三通道的卷积层为例，其对应的卷积计算的公式如（2-2）所示：

$$multiconv(IN_0, IN_1, IN_2, W_0, W_1, W_2, B) = B + \sum_{i=0}^{KernelSize} (IN_{0i} \times W_{0i} + IN_{1i} \times W_{1i} + IN_{2i} \times W_{2i}) \quad (2-2)$$

其中, IN_0 、 IN_1 、 IN_2 分别表示对应输入通道的特征值, W_0 、 W_1 、 W_2 分别表示对应输入通道的权重参数, B 表示偏置参数。

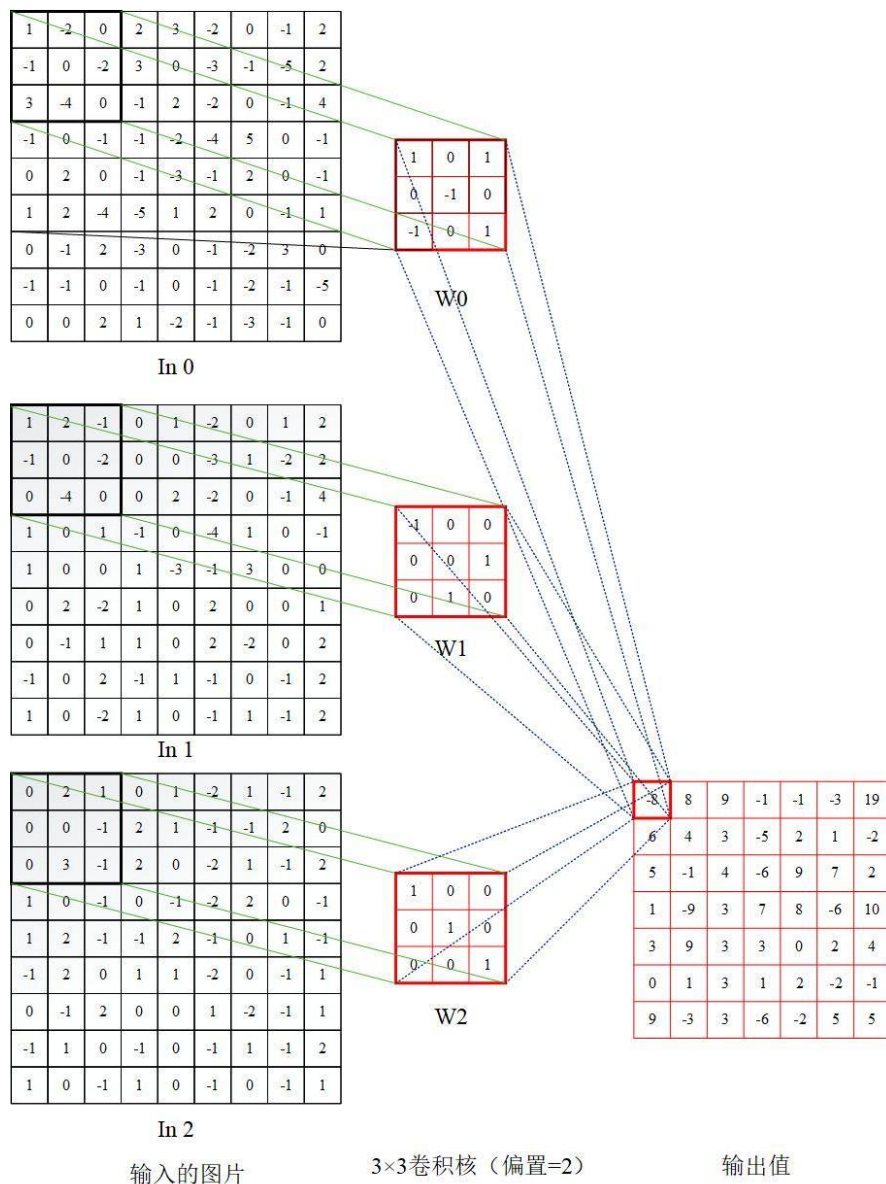


图 2-3 多通道卷积运算

Fig.2-3 Multi-channel convolution operation

按照公式 (2-2) 进行三输入通道的卷积计算, 具体过程如图 2-3 所示。卷积层的输入特征图的尺寸设置为 9×9 , 一共有三个通道 (In0、In1、In2), 单一通道的卷积核大小为 3×3 (偏置参数设置为 1), 步幅为 1, 卷积层的输出特征图的尺寸为 7×7 。按照公式 (2-2) 计算可以得出, 滑动窗口 (In0、In1、In2 特征图中的 3×3 黑色框)

的 9 个特征值分别与各自对应的 3×3 卷积核 (3×3 红色框 W_0 、 W_1 、 W_3) 的 9 个权重参数先按顺序进行乘运算, 再把乘运算的结果相加, 最后, 把三个通道得到的结果以及偏置参数相加在一起, 得到输出特征图的第一个特征值 (输出值中左上角 1×1 加深的红色框)。然后再移动滑动窗口, 依次进行卷积操作, 计算完成 7×7 输出特征图的全部特征值。

在图像处理任务中, 多个卷积核会生成多个特征图, 这些特征图堆叠在一起形成一个新的特征图, 卷积操作会改变输入数据的维度, 具体变化取决于卷积核的大小、步长和填充等参数。卷积操作前后的变化主要体现在数据的维度、特征表示、计算效率、稀疏性和不变性等方面。权值共享和局部连接特性显著减小计算复杂度和参数数量, 并通过多层堆叠的方式提取输入数据的层次化特征。这种高效的特征提取方式使得 CNN 能够在各种任务中取得优异的性能, 并成为深度学习领域的核心技术之一。

2.1.2 激活函数层

激活函数层是 CNN 和其他深度学习模型中的关键组成部分, 其主要作用是引入非线性特性, 使得神经网络能够学习和表示复杂的模式^[56]。如果没有激活函数, 无论神经网络有多少层, 其整体仍然是一个线性模型, 无法捕捉数据中的非线性关系。激活函数通过将输入信号转换为非线性输出, 使得神经网络能够拟合复杂的函数, 从而解决各种复杂的任务。常见的激活函数有 Sigmoid 函数、Tanh 函数和 ReLU 函数。

如公式 (2-3) 所示为 Sigmoid 激活函数, 如图 2-4 所示为 Sigmoid 激活函数曲线, 其输出值范围为 $[0, 1]$ 。

$$f(x) = \frac{1}{1 + e^{-x}} \quad (2-3)$$

其中, e^x 为指数函数, $e \approx 2.71828$, e^{-x} 为 e^x 的倒数。

Sigmoid 函数在现代深度学习中逐渐被其他激活函数所取代, 主要原因包括以下三个方面:

(1) 梯度消失问题: 当输入值非常大或非常小时, Sigmoid 函数的导数接近于 0, 导致梯度消失问题, 使得网络训练变得困难。

(2) 输出非零中心化: Sigmoid 函数的输出总是为正, 这会导致后续层的输入数据分布偏向正数, 从而影响梯度下降的效率。

(3)计算复杂度较高：Sigmoid 函数涉及指数运算，计算量较大。

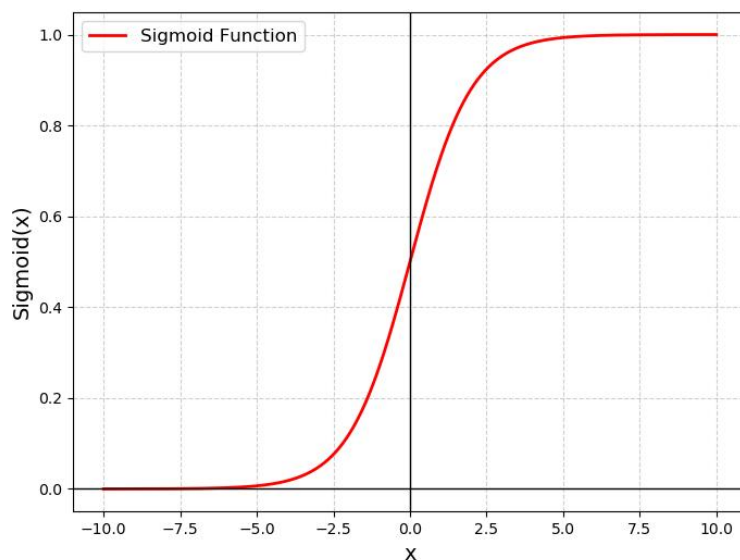


图 2-4 Sigmoid 激活函数曲线

Fig.2-4 Sigmoid activation function curve

如公式（2-4）所示为 Tanh 激活函数，如图 2-5 所示为 Tanh 激活函数曲线，其输出以 0 为中心，输出值范围为[-1, 1]。

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2-4)$$

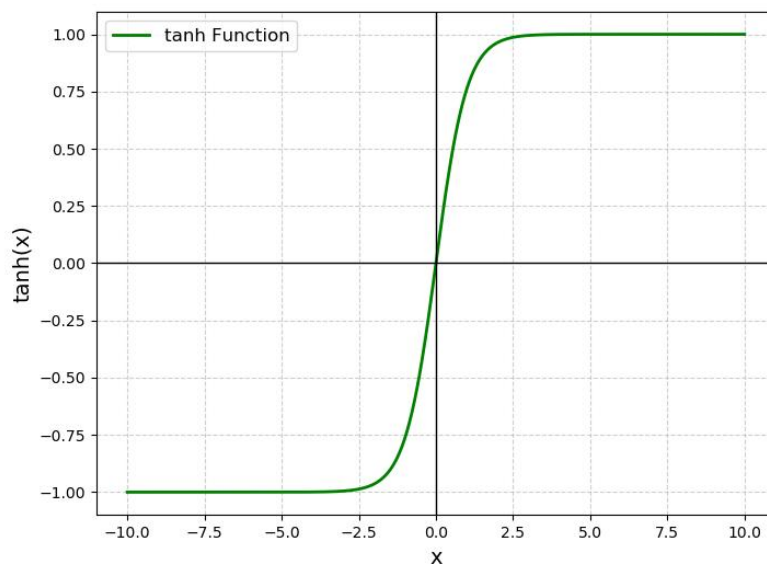


图 2-5 Tanh 激活函数曲线

Fig.2-5 Tanh activation function curve

Tanh 函数的输出以 0 为中心，这有助于加速梯度下降的收敛速度，与 Sigmoid

函数相比，Tanh 函数的梯度更强，能够缓解梯度消失问题。然而，Tanh 函数在输入值非常大或非常小时，其导数仍然会接近于 0，导致梯度消失。且 Tanh 函数同样涉及指数运算，计算量较大。梯度消失问题和计算复杂度较高的问题导致 Tanh 函数逐渐被 ReLU 等更高效的激活函数取代。

$$f(x) = \text{Max}(0, x) \quad (2-5)$$

如公式（2-5）所示为 ReLU 激活函数，如图 2-6 所示为 ReLU 激活函数曲线，其输出以 0 为中心，输出值范围为 $[0, +\infty]$ 。

ReLU 激活函数，也称为修正线性单元，其公式如（2-5）所示，其对应的函数曲线如图 2-6 所示。ReLU 函数将输入值中小于 0 的部分置为 0，大于 0 的部分保持不变。

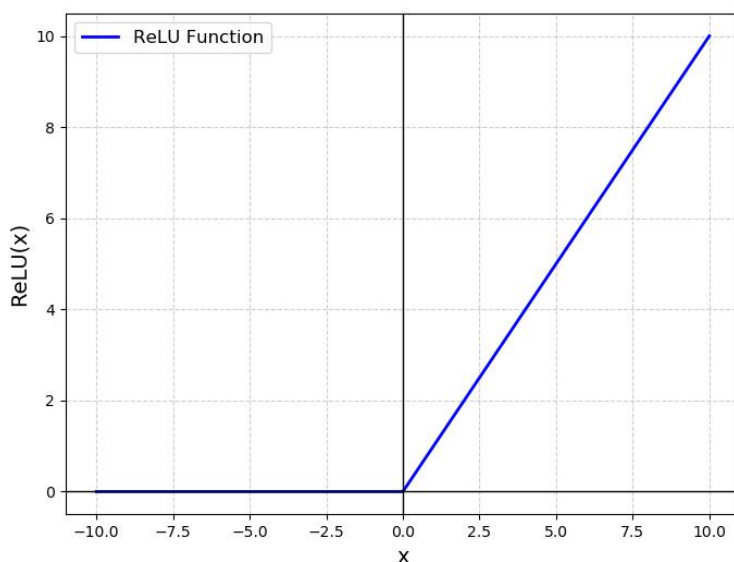


图 2-6 ReLU 激活函数曲线

Fig.2-6 ReLU activation function curve

与 Sigmoid 和 Tanh 激活函数相比，ReLU 函数成为现代深度学习中的主流激活函数的原因有：

- （1）计算简单：ReLU 函数只涉及简单的阈值操作，计算速度非常快。
- （2）缓解梯度消失问题：当输入值为正时，ReLU 函数的导数为 1，能够有效缓解梯度消失问题。
- （3）稀疏激活：ReLU 函数将负值置为 0，使得神经网络的激活变得稀疏，从而减少参数之间的依赖性，提高模型的泛化能力。

ReLU 激活函数也存在以下缺点：

(1) 神经元死亡问题：当输入值为负时，ReLU 函数的导数为 0，导致神经元无法更新参数，这种现象称为“神经元死亡”。如果大量神经元处于死亡状态，网络的表达能力会大大降低。

(2) 输出非零中心化：ReLU 函数的输出仍然是非零中心化的，可能影响梯度下降的效率。

为了克服 ReLU 函数的缺点，研究者也提出过多种改进版本，如：Leaky ReLU、Parametric ReLU (PReLU) 和 Exponential Linear Unit (ELU) 等。

2.1.3 下抽样层

下抽样层通常位于卷积层之后，用于对特征图进行下采样，从而减少数据的空间尺寸和计算量。池化层的主要作用是通过聚合局部区域的信息来降低特征图的分辨率，同时保留重要的特征信息。池化操作不仅能够减少模型的参数数量和计算复杂度，还能增强模型的鲁棒性和泛化能力。

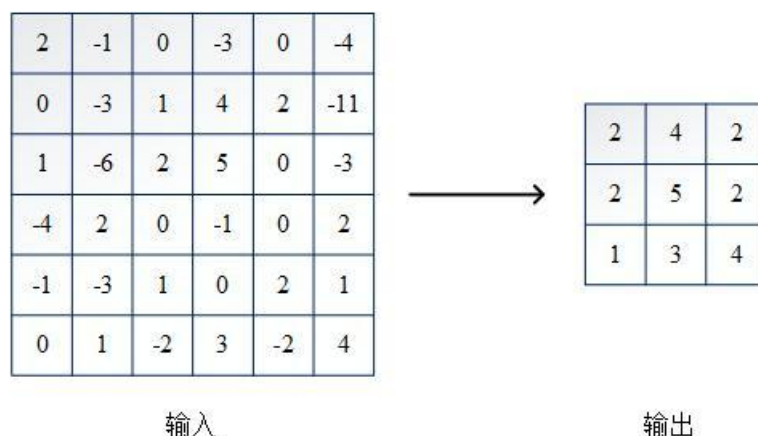


图 2-7 最大池化下抽样层计算

Fig.2-7 Maximum pooling downsampling layer calculation

如图 2-7 所示，池化层通过在输入特征图上滑动一个固定大小的窗口（称为池化窗口），并对窗口内的值进行聚合操作来实现下采样。池化窗口的大小和滑动步长是池化层的两个关键参数。常见的池化窗口大小为 2×2 ，步长为 2，这意味着每次池化操作会将特征图的高度和宽度减半。池化操作不涉及参数学习，因此是一种确定性的计算过程。

2.1.4 全连接层

在全连接层中，每个神经元与前一层的所有神经元相连，因此称为“全连接”。

全连接层的作用是将高维特征映射到目标类别空间，例如在图像分类任务中，将提取的特征映射到类别概率分布。全连接层的输出通常通过一个 Softmax 函数进行归一化，得到每个类别的概率值。如图 2-8 所示为全连接层的拓扑结构。

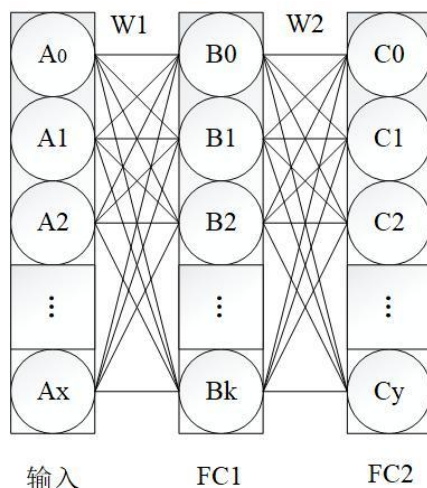


图 2-8 全连接层拓扑结构

Fig.2-8 Fully connected layer topology

从图 2-8 中可以看出，输入层有 x 个神经元（表示为 $A=[A_0, A_1, A_2, \dots, A_x]$ ），FC1 层有 k 个神经元（表示为 $B=[B_0, B_1, B_2, \dots, B_k]$ ），FC2 层有 y 个神经元（表示为 $C=[C_0, C_1, C_2, \dots, C_y]$ ）。FC1 层的每个神经元接收所有输入 $A=[A_0, A_1, A_2, \dots, A_x]$ ，并通过加权求和（权值参数 W_1 ）和激活函数生成输出 $B=[B_0, B_1, B_2, \dots, B_k]$ 。FC2 层的每个神经元接收 FC1 层的所有输出 $B=[B_0, B_1, B_2, \dots, B_k]$ ，并通过加权求和（权值参数 W_2 ）和激活函数生成输出 $C=[C_0, C_1, C_2, \dots, C_y]$ 。FC1 层和 FC2 层的计算公式分别如（2-6）和（2-7）所示：

$$B = Bias_{FC1} + \sum_{i=0}^x W_{1i} \times IN_i \quad (2-6)$$

其中， B 为 FC1 层的输出特征值， $Bias_{FC1}$ 为 FC1 层的偏置参数， W_{1i} 为 FC1 层的权重参数， IN_i 为 FC1 层的输入特征值。

$$C = Bias_{FC2} + \sum_{i=0}^k W_{2i} \times B_i \quad (2-7)$$

其中， C 为 FC2 层的输出特征值， $Bias_{FC2}$ 为 FC2 层的偏置参数， W_{2i} 为 FC2 层的权重参数， B_i 为 FC2 层的输入特征值。

卷积层的卷积操作和池化层的池化操作本质上都是可以并行执行的，每个卷积

核和池化窗口都可以独立地在输入数据的不同位置进行计算，这种并行性非常适合在硬件（如 GPU、FPGA）上实现，能够显著加速计算过程。此外，通过局部连接特性减少参数数量，降低计算复杂度。权值共享进一步减少参数数量和存储需求，使得 CNN 在硬件实现时能够高效地复用计算资源。CNN 的计算过程可以分解为多个阶段，这些阶段可以通过流水线设计在硬件上并行执行，从而提高吞吐量。通过合理的数据布局和内存访问模式，可以减少数据搬运的开销，提高计算效率。

2.2 经典网络模型 LeNet-5

LeNet-5 是由 Yann LeCun 等人于 1998 年提出的一种经典的卷积神经网络(CNN)模型，主要用于手写数字识别任务（如 MNIST 数据集）。如图 2-9 所示，LeNet-5 的网络结构由 7 层组成，包括 2 个卷积层、2 个池化层和 3 个全连接层。

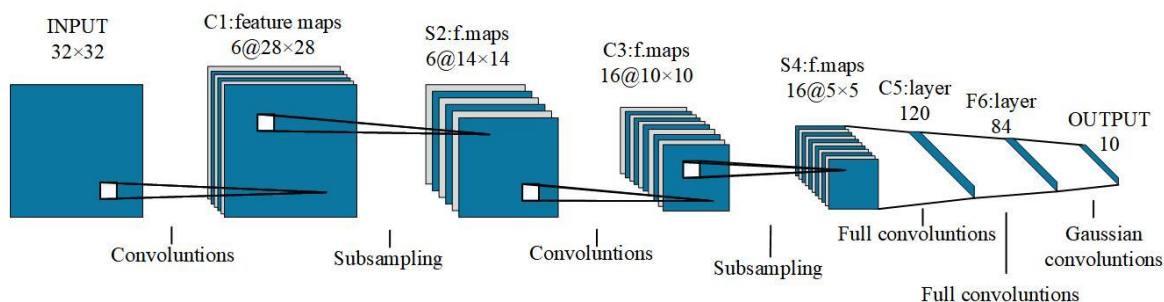


图 2-9 LeNet-5 卷积神经网络

Fig.2-9 LeNet-5 convolutional neural network

以下是每一层的详细说明：

1) 输入层：输入数据的尺寸为 32×32 的灰度图像。虽然 MNIST 数据集的图像尺寸为 28×28 ，但 LeNet-5 将输入图像填充到 32×32 ，以便更好地提取边缘特征。

2) C1 卷积层：输入尺寸为 $32 \times 32 \times 1$ （高度 $\times$ 宽度 $\times$ 通道数），使用 6 个 5×5 的卷积核，步长为 1，无填充，输出尺寸为 $28 \times 28 \times 6$ （每个卷积核生成一个特征图）。参数数量共计 $6 \times (5 \times 5 + 1) = 156$ 个。乘运算共计 117600 次。

3) S2 池化层：输入尺寸为 $28 \times 28 \times 6$ ，使用 2×2 的最大值池化，步长为 2，输出尺寸为 $14 \times 14 \times 6$ 。

4) C3 卷积层：输入尺寸： $14 \times 14 \times 6$ ，使用 16 个 5×5 的卷积核，步长为 1，无填充，输出尺寸： $10 \times 10 \times 16$ 。参数数量共计 $16 \times (5 \times 5 \times 6 + 1) = 2416$ 个。乘运

算共计 240000 次。

5) S4 池化层：输入尺寸为 $10 \times 10 \times 16$ ，同样使用 2×2 的最大值池化，步长为 2，输出尺寸为 $5 \times 5 \times 16$ 。

6) F5 全连接层：输入尺寸为 $5 \times 5 \times 16 = 400$ ，将 S4 池化层的输出展平为一个 400 维的向量，并通过 120 个神经元进行全连接，故输出尺寸为 120。参数数量共计 $400 \times 120 + 120 = 48120$ 个。乘运算共计 48000 次。

7) F6 全连接层：输入尺寸为 120，输出尺寸为 84。参数数量共计 $120 \times 84 + 84 = 10164$ 个。乘运算共计 10080 次。

8) 输出层：输入尺寸为 84，包含 10 个神经元，对应 MNIST 数据集的 10 个类别（0-9），输出尺寸为 10。参数数量共计 $84 \times 10 + 10 = 850$ 。乘运算共计 840 次。

LeNet-5 在深度学习发展史上具有开创性和启发性意义，为后续的深度学习模型奠定了基础。尽管存在一些局限性，但 LeNet-5 在深度学习领域具有很大的研究价值。

2.3 卷积神经网络加速方法

2.3.1 模型轻量化技术

模型轻量化^[57]是把 CNN 部署在资源受限设备上的重要发展方向。CNN 通常包含大量的卷积操作和全连接层，导致计算复杂度极高，如图 2-10 所示，轻量化技术通过减少模型的计算量，使其能够在计算能力有限的设备上高效运行。

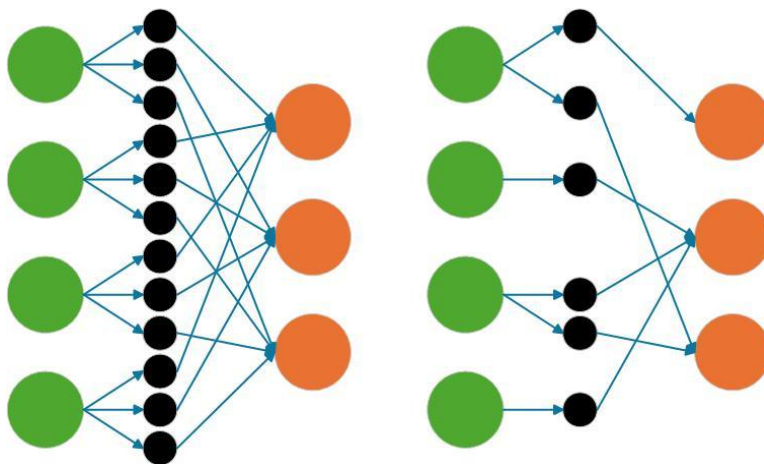


图 2-10 轻量化示意图

Fig.2-10 Lightweight schematic diagram

大规模模型需要存储大量的参数和中间计算结果，这对设备的存储空间提出了较高要求。在边缘计算场景中，设备存储空间有限，轻量化技术通过压缩模型参数

和优化存储结构，可以减少内存占用，降低对本地存储的需求。此外，轻量化技术还可以通过减少计算量和数据搬运，降低设备的功耗，延长电池寿命。在边缘计算中，降低功耗有助于延长设备的续航时间，减少维护成本。更重要的是轻量化模型通常具有更少的参数和更简单的计算结构，能够显著提高推理速度，满足实时性要求较高的应用场景。这在边缘计算实时处理任务时，能及时响应，无需等待云端反馈。

模型轻量化主要通过模型剪枝^[58]、量化^[59]和知识蒸馏^[60]等方法实现。模型剪枝通过移除模型中权重值接近零的参数或移除对输出贡献较小的神经元，可以减少模型参数和计算量，同时保持较高的精度。量化通过把模型中的权重值从高精度转换为低精度（如将 32 位浮点数转换为 8 位整数），可以减少内存占用和计算量，同时保持较高的推理精度。知识蒸馏则是通过训练一个小型模型来模仿一个大型模型的行为，从而将大型模型的知识转移到小型模型中，目的是在减少模型规模的同时，保持较高的精度。

综上所述，模型轻量化的目的是通过减少模型的复杂度、参数量和计算量，使其更适合在资源受限的设备上部署和运行。轻量化技术主要包括模型剪枝、量化和知识蒸馏等方法。这些方法在降低计算复杂度、减少内存占用、降低功耗和提高推理速度方面具有显著优势，能够满足移动设备、嵌入式系统和边缘计算等场景的需求。随着深度学习技术的不断发展，模型轻量化将在更多实际应用中发挥重要作用。

2.3.2 硬件加速技术

硬件加速技术主要依赖通用加速器或专用加速器实现，各自针对不同的计算需求进行优化。如表 2-1 所示，通用加速器采用高度灵活的架构设计，能够适应广泛的计算任务。CPU 基于通用指令集，擅长处理逻辑控制、分支预测等串行任务，但并行计算能力有限；而 GPU 凭借大规模并行计算单元，在深度学习训练、图形渲染等数据密集型任务中表现出色。通用加速器的优势在于高灵活性和成熟的软件生态，开发者可以快速部署和优化算法，但其能效比和计算效率较低，难以满足低功耗或高实时性场景的需求。

相比之下，专用加速器针对特定计算任务（如 CNN 推理）进行硬件级优化。ASIC 通过固化算法逻辑实现极高的计算效率和极低功耗，在 AI 推理任务中的性能远超通用 GPU；FPGA 则提供可重构硬件电路，允许用户根据需求调整计算架构，在通信、自动驾驶等领域广泛应用。

表 2-1 通用加速器与专用加速器优缺点对比

Tab.2-1 Comparative analysis of general-purpose and special-purpose computing accelerators

	通用加速器 (GPU/CPU)	专用加速器 (ASIC/FPGA)
计算效率	存在冗余开销	并行度高、延迟低
功耗	GPU 高并行计算需大功耗 CPU 串行效率低	无效功耗减少
开发成本	GPU 可直接部署，生态成熟	ASIC 需定制流片 FPGA 开发周期长
适用场景	云端训练、多任务推理、算法快速迭代	边缘计算、嵌入式设备

在专用加速器领域，ASIC 和 FPGA 是两种主流的硬件实现方式，各自在性能、灵活性和成本等方面具有显著差异。ASIC 是为特定算法或计算任务定制的芯片，其硬件逻辑一旦制造完成即固定不可更改。ASIC 的核心优势在于极高的计算效率和超低功耗，由于完全针对目标算法优化，其性能通常远超通用处理器（如 CPU/GPU）甚至 FPGA。然而，ASIC 的研发周期长，流片成本极高，且无法适应算法迭代。一旦算法变更，ASIC 可能面临淘汰风险，因此仅适合算法成熟、需求稳定的场景。

FPGA 则通过可编程逻辑单元和硬件描述语言（如 Verilog）实现算法硬件化，其最大特点是可重构性。用户可根据需求动态调整电路结构，从而快速适配不同算法。FPGA 在原型验证和中小批量生产场景中极具价值，尤其适合算法尚未固化或需要频繁更新的领域。FPGA 的缺点主要是开发门槛高，单位成本高，且峰值性能受限于芯片规模和时钟频率。

综合来看，ASIC 适合算法固定、追求极致性能与功耗的大规模应用，而 FPGA 更适合算法迭代频繁或需要硬件灵活性的场景。

2.3.3 软件框架优化技术

软件框架优化是提升深度学习系统整体效能的核心技术体系，它通过系统化的代码转换和资源管理策略，在保持算法逻辑不变的前提下，最大限度地释放硬件计算潜力。这一技术领域处于算法与硬件的交汇点，既要理解神经网络的计算特性，又要掌握底层硬件架构的细节，是现代 AI 基础设施不可或缺的组成部分。

在计算图层面的优化是深度学习软件框架中的核心优化技术，主要针对神经网络的计算图表示进行系统性的改造与增强。计算图优化的本质是通过图变换技术，在不改变模型数学等价性的前提下，提升计算效率、减少内存占用并优化硬件利用率。这一过程通常发生在模型训练或推理之前，属于静态优化的范畴。计算图优化包含多层次的技术策略。在基础层面，算子融合是最常见且有效的优化手段，它将多个连续操作合并为单个复合操作。在更复杂的优化层面，常量折叠会在编译时预先计算静态子图，消除运行时开销。

在编译器层面的优化是连接高层算法描述与底层硬件指令的关键桥梁，专注于将前端框架生成的中级表示转换为高度优化的机器代码。与传统编译器不同，深度学习编译器需要处理张量计算的特有模式，并适应多样化的硬件架构。硬件特定优化是编译器的关键能力。对于 GPU，编译器会优化线程块配置、共享内存使用和寄存器分配，最大化利用架构的并行性。针对 TPU 等专用加速器，编译器需要精确映射到硬件原语，如将矩阵乘法匹配到脉动阵列单元。内存优化方面，编译器实施缓冲区合并、数据布局转换等策略，确保数据访问模式与硬件特性匹配。特别重要的是量化支持，编译器需要智能处理从浮点到定点表示的转换，插入适当的缩放和舍入操作，同时保持模型精度。

2.4 本章小结

本章从神经网络的理论基础出发，以 CNN 作为多层神经网络的典型代表，详细分析了 CNN 的拓扑结构（主要包括卷积模块、激活模块、池化模块和全连接模块）。然后以 LeNet-5 为例，分析了该模型的 7 层结构的输入输出特征图、基础运行理论、权重与偏置参数总量和乘运算次数等，并对 CNN 相关的模型轻量化技术、硬件加速技术和软件框架优化技术进行了介绍，为第三、四章节中设计的基于 FPGA 加速的 CNN 轻量化模型奠定了理论基础。

3 基于 Zynq 的 CNN 目标识别算法设计与实现

在第二章的理论研究基础上,开展了基于 Zynq 的 CNN 目标检测算法研究与设计,在本章中主要进行了 CNN 模型的轻量化设计,并实现其在 Zynq 上的部署。以 LeNet-5 模型为基础,提出一种对 CNN 轻量化设计的思路,并完成对 CNN 架构的轻量化设计。然后,在 Zynq 上完成改进后模型的搭建,利用 Zynq 的特性实现对 CNN 各个模块的加速,对仿真结果进行分析,实现对 CNN 目标检测算法的加速。

3.1 CNN 架构轻量化设计

CNN 主要包含卷积层、池化层、激活层和全连接层四种基本结构单元。卷积层是 CNN 所特有的一种网络结构,主要通过滑动窗口对应的图像数据与权值参数相卷积的方式对数据样本进行特征提取;池化层又被称为降采样层,一般与卷积层相结合,使用池化函数对卷积层的输出特征数据进行简化,降低特征图像的参数;全连接层是 CNN 的输出层,作用是对图像的特征进行分类,通过对前一层提取的特征进行非线性组合,从而达到学习的目的。

CNN 的训练过程由前向和反向传播阶段组成。在本章中,决定把 CNN 的前向传播阶段部署在 Zynq 的 PL 端,在满足 CNN 模型预准确率的情况下,对 CNN 的架构进行了优化,目的是减少模型的参数总量和计算量。基于 CNN 的基本结构和训练原理,对部署在 Zynq 上的 CNN 框架进行了如下设计:

- (1) 卷积层统一使用 3×3 的卷积核,步长为 1,为了不降低分类性能,每经过两个卷积层才会通过池化层进行特征简化;
- (2) 池化层均使用最大池化,相比于平均池化,更容易部署在 Zynq 平台上;
- (3) 激活函数选用 ReLU 函数,与常见的 Sigmoid、Tanh 等激活函数相比,更容易实现;
- (4) 全连接层使用 Softmax 函数进行分类,本章架构只使用一个全连接层实现图像分类,降低参数量。

本系统的 CNN 的前期训练是在 Python 上完成的,其前向传播的框架主要由 4 个卷积层、5 个激活层、2 个池化层和 1 个全连接层组成。如图 3-1 所示。把本系统搭建在 Zynq 上,预测准确率达到了 98.2%。

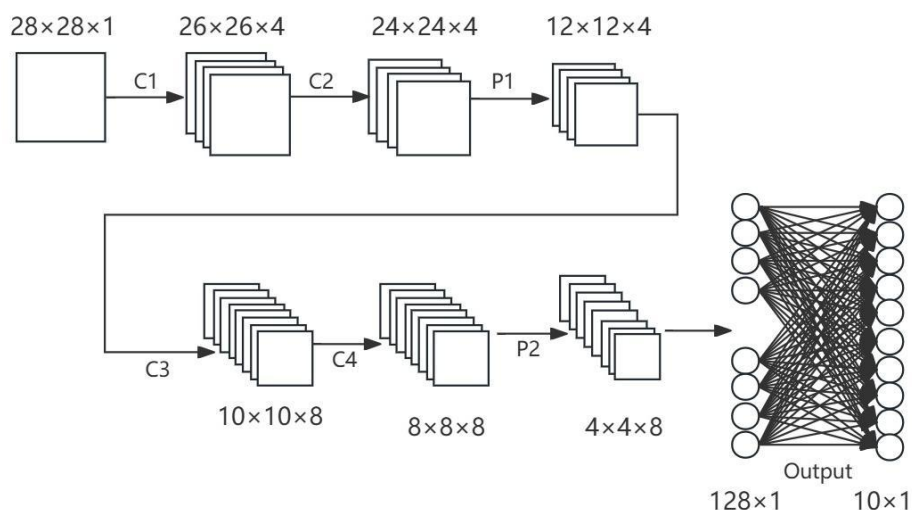


图 3-1 卷积神经网络轻量化模型

Fig.3-1 Lightweight models of convolutional neural networks

本章的模型输入数据大小为 $28 \times 28 \times 1$ ，4 个卷积层（C1，C2，C3，C4）、2 个池化层（P1，P2）、1 个输出层（Output），输出 0~9 共 10 个分类结果。该 CNN 模型每层的输入大小、输出大小、卷积核大小和参数分布如表 3-1 所示：

表 3-1 卷积神经网络模型参数统计

Tab.3-1 Statistics of convolutional neural network model parameters				
	输入	输出	卷积核大小	参数量
C1	$28 \times 28 \times 1$	$26 \times 26 \times 4$	$3 \times 3 \times 4$	36+4
C2	$26 \times 26 \times 4$	$24 \times 24 \times 4$	$3 \times 3 \times 4$	144+4
P1	$24 \times 24 \times 4$	$12 \times 12 \times 4$	/	/
C3	$12 \times 12 \times 4$	$10 \times 10 \times 8$	$3 \times 3 \times 8$	288+8
C4	$10 \times 10 \times 8$	$8 \times 8 \times 8$	$3 \times 3 \times 8$	576+8
P2	$8 \times 8 \times 8$	$4 \times 4 \times 8$	/	/
Output	128	10	1×1	1280+10
合计	2324+34=2358			

把 LeNet-5 的 5×5 卷积核替换为 3×3 可以大幅减少内存占用和计算量，也会造成精度的损失，需要采取一定的措施进行弥补。本章采取的措施主要包括以下两点：

（1）增加网络深度： 5×5 卷积核替换为 3×3 后，导致感受野减小，模型的感知能力下降。增加网络的深度可以扩大感受野，LeNet-5 只有两个卷积层，通过把卷积层增加到四个，弥补了感受野减小的问题。

（2）调整量化策略：在 Zynq 的 PL 端部署 CNN 模型，一般都需要把浮点数转

化为定点数，参数整型化能有效节省硬件设备的资源，通常来说，模型的参数精确度越高，预测的准确度也会相应地有所提高，所占用的资源自然也会越多。提高量化区间，可以补偿 CNN 模型损失的精度。如表 3-1 所示，该模型参数总量为 2358，经典的 LeNet-5 模型有近 5 万个参数，本模型仅占其参数总量的 5%，所以，提高参数的量化区间并不会占用太多内存。

本章主要通过降低卷积核尺寸、增加网络深度和采用 Int14 型数据的措施完成对 LeNet-5 模型的轻量化，乘加运算消耗的片上资源在开发板的允许范围之内，该模型的识别准确率为 98.2%，精度损失的程度很小，与其它的轻量级 CNN 模型相比，识别准确率还有一定的提升。

3.2 系统总体框架设计

该模型主要部署在开发板的 PL 端，如图 3-2 所示，在卷积模块前添加了一个降采样模块，用于模拟对图像数据的预处理。把大小 28×28 的图像数据按照 784×1 的尺寸送到卷积模块的输入端口。本设计使用 RAM IP 核存储特征图像的数据，使用 ROM IP 核存储卷积层和全连接层的参数，这样的配置，既便于实时读写特征图像数据，又能保证卷积和全连接层参数的稳定性，降低数据丢失风险，提升系统运行的可靠性。最后，根据全连接层的输出来确定图像分类的结果。

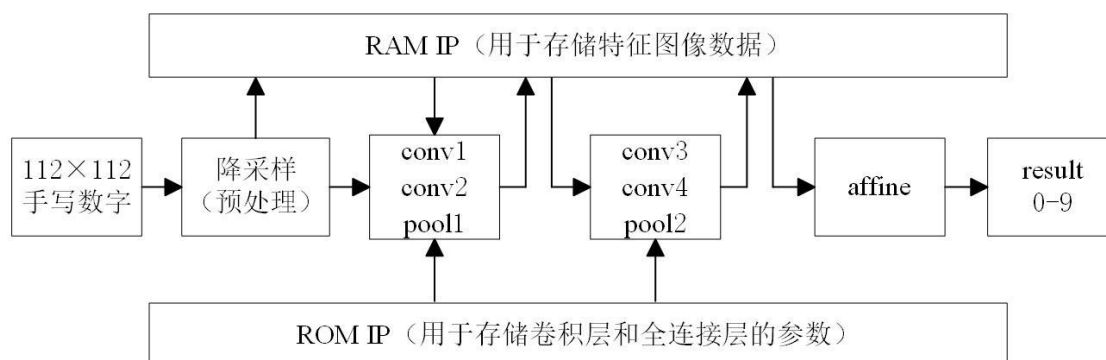


图 3-2 系统整体框图

Fig.3-2 System overview block diagram

3.3 模块加速设计

3.3.1 卷积模块加速设计

卷积运算是一种二维的乘加运算 (MAC)，它是由多个乘、加运算组合而成的。

实现这种卷积运算的结构有很多, 比如 Farabet 等人提出的卷积硬件实现方法, 如图 3-3 所示, 这种 3×3 的卷积窗口充分利用了卷积运算的并行性。以第二卷积层为例, 该模块只设置一个输入端口接收特征数据, 所以本文设计通过三个行寄存器存储数据, 并实现卷积窗口的滑动, 利用并行结构加流水线操作实现高效的卷积计算电路。当寄存器组中存储到第三行第三个特征图像数据的时候, 在接下来的每一个时钟周期里都会完成一次 4 个滤波器通道对应的 3×3 的卷积窗口滑动的乘加运算, 并输出每个通道对应的卷积结果。只需要输入一遍特征图像数据即可完成该层的卷积运算, 在 50MHz 的频率下工作, 从接收第一个图像数据到输出最后一个卷积运算结果只需要 $39\mu\text{s}$, 与串行计算电路相比, 加速效果明显。

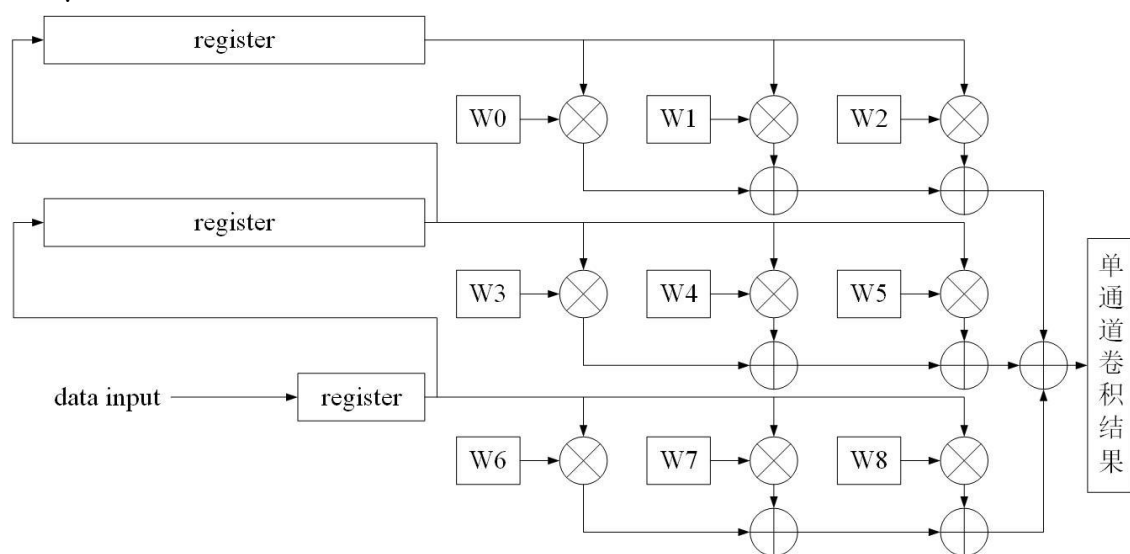


图 3-3 卷积计算电路图

Fig.3-3 Convolution calculation circuit diagram

3.3.2 池化模块加速设计

池化模块的输入数据是上一卷积层输出的特征图像数据按照行顺序单端口传输而来。最大池化函数需要从 2×2 的图像特征中输出最大值, 本文主要利用输入数据的奇偶行、奇偶列来比较大小, 进而找到其中的最大值。首先是奇数行, 每两个数据进行比较, 用 FIFO 对较大者进行缓存; 到偶数行时, 每两个数据进行比较, 再与上一个奇数行相同列数据的较大者进行比较, 从而达到池化的目的, 如图 3-4 所示。

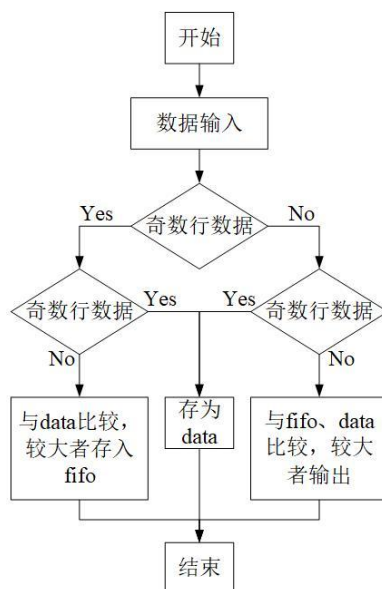


图 3-4 最大池化流程图

Fig.3-4 Flowchart for maximum pooling

3.3.3 卷积+池化模块的流水线设计

在资源允许的情况下, 在卷积层的内部使用并行性结构的同时, 也可以在两个相邻的卷积层之间使用流水线操作。当第一层卷积计算结果经过 ReLU 函数激活以后, 会传输到相邻的第二卷积层, 第二卷积层接收到特征图像数据以后就可以开始工作, 当该层计算出卷积结果后, 会直接传输到池化层, 完成最大池化后的特征图像数据才会存储到 RAM IP 核中。这种卷积+池化的流水线设计不仅可以节省数据存储占用的资源, 还可以减少储存数据产生的时延, 进一步提高整体架构的实时性。

3.3.4 全连接模块设计

本文设计的框架只包含一个全连接层, 它的输入为 128×1 、输出为 1×10 、卷积核尺寸为 1×1 。全连接层和卷积层一样需要进行乘加运算, 具有参数使用量多的特点。如图 3-5 所示, 本文采用 10 通道并行乘加运算的策略。该模块从 RAM IP 核中按照 128×1 的形式读取特征图像数据, 并从 ROM IP 核中对应的读取参数, 每一个时钟周期会进行 10 次 1×1 的卷积运算, 每个通道的卷积结果分别进行加运算。从数据输入开始计算, 全连接层仅需要 129 个时钟周期得到各个通道的计算数据, 并在接下来的几个时钟周期内判断出最终的分类结果。

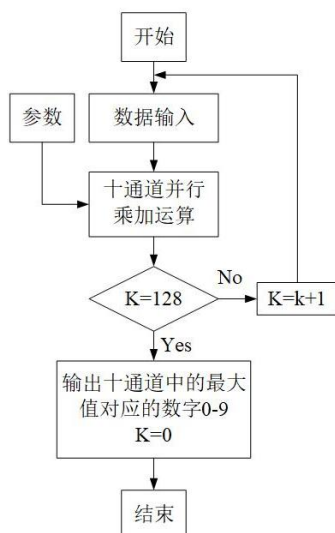


图 3-5 全连接模块计算流程图

Fig.3-5 Flowchart for fully connected module computation

3.4 基于 Zynq 平台搭建卷积神经网络系统

本章决定选择 Xilinx XC7Z020-2CLG400I 作为硬件平台，主要利用该平台的可编程逻辑（PL）来完成对 CNN 轻量化模型的搭建。

在 Zynq 平台上搭建 CNN 系统，除了完成各模块方案设计之外，还需要解决读写图像数据地址、数据存储方案和设计流水线结构等关键问题。首先根据 Zynq 平台 XC7Z020-2CLG400I 的硬件特性对实现 CNN 进行方案分析，其次通过分析硬件特性、提出解决方案并完成设计与实现，最后在 Zynq 平台上实现一个高效、轻量化的 CNN 系统。

3.4.1 Zynq 平台实现卷积神经网络方案分析

首先，CNN 需要对输入图像进行逐层处理，因此需要设计高效的地址生成方案，以确保数据能够正确读取，图像数据读取地址生成方案主要是通过卷积核的大小和步长，动态生成图像数据的读取地址。

其次，Zynq 的存储资源有限，尤其是片上 Block RAM 的容量较小（XC7Z020-2CLG400I 的 Block RAM 容量为 4.9Mbit）。Block RAM 的调用方便且读写速度快，适合存储中间数据。然而，DDR3 容量较大（约 1Gbit），调用麻烦且读写速度较慢，因此主要用于存储不频繁访问的数据。CNN 的各层之间需要高效的数据传输，故本章主要通过 Block RAM 作为该 CNN 模型的中间存储器，在卷积层

和池化层之间进行数据传输。

最后，CNN 的计算过程需要高效的数据流动，以避免计算瓶颈。本章将通过流水线设计，将卷积层和池化层的计算过程并行化，既能发挥嵌入式平台的并行计算优势，又可以提高系统的运行速度。CNN 的计算任务往往需要在 Zynq 的 PL 和 PS 之间协同完成，本章计划通过硬件-软件协同设计，将计算密集型任务（如卷积计算）分配给 PL，而逻辑控制任务分配给 PS。

本章设计的轻量化 CNN 模型各模块需要存储数据的大小如表 3-2 所示。

表 3-2 卷积神经网络各模块参数大小

Tab.3-2 Parameter sizes of each module in a convolutional neural network

	特征参数/bit	权重参数/bit	偏置参数/bit
Input	$28 \times 28 \times 1$		
C1	$26 \times 26 \times 4 \times 24$	$3 \times 3 \times 4 \times 16$	4×16
C2	$24 \times 24 \times 4 \times 40$	$3 \times 3 \times 4 \times 16$	4×16
P1	$12 \times 12 \times 4 \times 40$		
C3	$10 \times 10 \times 8 \times 56$	$3 \times 3 \times 4 \times 8 \times 16$	8×16
C4	$8 \times 8 \times 8 \times 72$	$3 \times 3 \times 8 \times 8 \times 16$	8×16
P2	$4 \times 4 \times 8 \times 88$		
F	128×104	$128 \times 10 \times 16$	$1 \times 10 \times 16$

在一般情况下，CNN 的池化层与卷积层是交替出现的，相邻的卷积模块和池化模块之间都需要设置一个 Block RAM 存储器，用来存储中间通道的特征值。而本章搭建的轻量化 CNN 是每隔两个卷积层出现一个池化层，故连续的两个 CNN 之间可以不对中间数据进行存储，只在卷积层和池化层之间加入 Block RAM 进行中间数据存储。一方面，加深的 CNN 可以避免 Block RAM 存储数据的次数增加，一定程度上控制住了 Block RAM 片上存储资源的使用量；另一方面，该模型中的相邻卷积层之间可以被设置成“卷积层+ReLU+卷积层+ReLU”形式的流水线，避免在中间模块添加 Block RAM 存储器在存储特征值上造成的时间浪费，节省系统运行的时间，达到提高整体模型的运行速度。

3.4.2 读写地址生成规律

做为 CNN 中的核心操作，卷积计算的滑窗操作主要用于从输入特征图中提取局部特征，滑窗操作通过卷积核在输入特征图上滑动，逐位计算局部区域的加权和，

从而生成输出特征图。滑窗操作如图 3-6 所示：

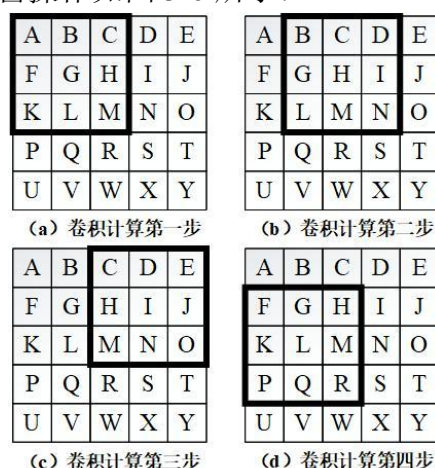


图 3-6 卷积计算原理示意图

Fig.3-6 Schematic diagram of convolution calculation principle

在卷积计算进行之前，需要先把图像数据按序进行存储，图像数据在 Block RAM 中是按行存储的，每行的数据地址连续，上一行的最后一个数据与下一行的第一个地址是连续的。本章使用的是 3×3 的卷积核，需要读取 9 个特征数据。图像数据按照行顺序进行读写，这意味着滑动窗口内对应特征值的地址不连续。如图 3-6 所示，以卷积核的第一个 3×3 滑动窗口为例，第一行 A、B、C 数据依次对应的地址为 0、1、2；第二行 F、G、H 数据依次对应的地址为 5、6、7；第三行 K、L、M 数据依次对应的地址为 10、11、12。滑动窗口内的 9 个特征值地址之间也存在着一定的规律。假设输入图像的尺寸为 $W \times W$ ，在生成特征值存储地址的过程中，第 n 行的第 m 个图像特征值的地址可以表示为列数 m 乘上行数 n ，然后减 1，具体计算方式如公式 (3-1) 所示。基于这一规律，可以设计一个高效的地址生成器，动态生成扫描窗口内每个特征值的地址，从而提高卷积计算的效率。

$$Address = m * n - 1 \quad (3-1)$$

其中， $Address$ 为存储数据的地址， m 为特征图的行数， n 为特征图的列数。

卷积计算模块的输入和输出特征图生成特征值寄存地址均可以使用以上方式。卷积计算过程中还要考虑“滑动窗口”的移动，本章主要通过三个移位寄存器来实现该项操作。具体思路如下，可以首先确定图像的尺寸，以 5×5 大小的图像为例，可以设计先两个可按位存储 5 位图像数据的移位寄存器（r1、r2），再设计一个可按位存储 3 位图像数据的移位寄存器（r3），数据读取地址生成结束之后，第一行的 5 个数据的按位存入到 r1 寄存器中，第二行的 5 个数据的按位存入到 r2 寄存器中，第

三行的 3 个数据的按位存入到 r3 寄存器中，此时，第一次卷积所需要的 9 位数据已经按照相同的行列顺序存入移位寄存器，如图 3-7 (a) 所示。计算完成后，按照图 3-7 (b) 所示进行一次移位即可进行第二次卷积运算，同理完成下一次卷积运算，如图 3-7 (c) 所示。当第一行卷积运算全部完成后，依次读取 3 次数据并按序进行 3 次移位即可计算下一行的第一个卷积（读取数据的次数与卷积核的尺寸相对应），如图 3-7 (d) 所示。当按照顺序把 5×5 图像的 25 位数据全部读取完毕时，即对应着最后一个卷积运算，这样做的好处是避免了对数据的二次读取，且对按序存储的数据进行读取的最佳方式就是以同样的顺序进行读取。

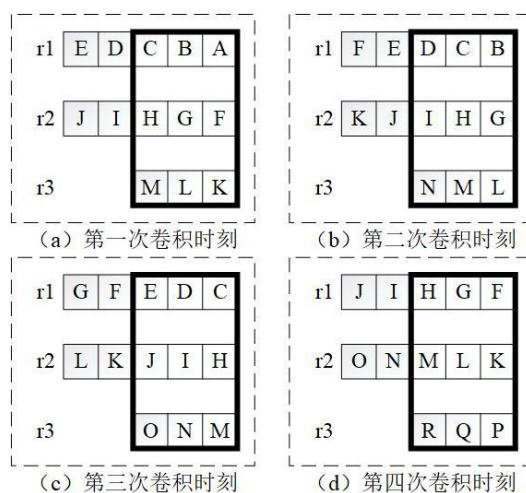


图 3-7 读取数据原理图

Fig.3-7 Schematic diagram of data reading principle

3.4.3 存储方案选择及流水线设计

在 CNN 的硬件实现中，存储方案选择 和 流水线设计 是确保系统高效运行的关键环节。CNN 层间会生成大量的中间数据（特征图），这些数据的存储是一个重要问题。移位寄存器无法满足大数据量的存储需求，使用 DDR3 存储器会使整个系统变得复杂，并产生较高延迟，故需要选择一种高效的存储方案。Block RAM 的读写速度远高于 DDR3，适合频繁访问的中间数据存储。Block RAM 可以通过硬件描述语言（如 Verilog）直接调用，适合在 Zynq 平台上实现。Block RAM 作为中间数据的存储器主要用于池化模块之后。

按照之前设计的地址生成方案，从 Block RAM 中读取中间数据。例如，对于一个 3×3 的卷积核，需要读取 9 个特征数据，这些数据可以通过移位寄存器快速读取，按照正常顺序将计算结果写入 Block RAM 中。例如，在每次池化操作之后，将计算

结果存储到 Block RAM 中。

从 Block RAM 中读取特征图数据,进行卷积计算。如图 3-8 所示,原图经过 Conv1 和 Conv2 后,进行 Pool1 操作,然后将结果存储到 Block RAM 中。从 Block RAM 中读取数据,进行 Conv3 和 Conv4 操作,然后进行 Pool2 操作,再将结果存储到 Block RAM 中。

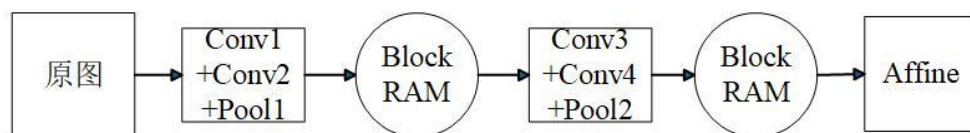


图 3-8 CNN 模型流水化设计图

Fig.3-8 Pipelined design diagram of CNN model

通过在两个功能单元之间加入寄存器,可以提高系统的处理速度。这种设计属于“面积换性能”的情况,即通过增加存储空间(Block RAM)来换取处理速度的提升。流水线设计的优势在于,通过并行化计算过程,减少数据读取和计算的延迟。通过合理分配 Block RAM 的存储空间,优化硬件资源的利用。当 Conv1 模块在计算原始图片时,会把得出的结果直接送给 Conv2 层,并非是在 Conv1 层计算全部完成后才会开始 Conv2 层的计算,对 Conv2 层的计算结果进行简单的加和运算,并产生 4 张不同的特征图,2 张特征图会分别通过 4 个不同的通道进入 Pool1 模块进行池化,池化后的特征图会按照前面的数据存储方式进行存储和读取。后续的 Conv3、Conv4 和 Pool2 模块也类似于前面的 Conv1、Conv2 和 Pool1 组成流水线设计,并对 Pool2 输出的数据进行存储。然后, Affine 层的输出结果为 10 个具体的特征值,根据这十个特征值的大小和正负即可判断出该 CNN 的预测结果。

本章使用的 Block RAM 寄存器成功把“Conv1+Conv2+Pool1”和“Conv3+Conv4+Pool2”这两个流水线设计串联在了一起,也保证了各个模块之间的特征数据的有效且准确的传输,Block RAM 寄存器的插入极大地提高了整个模型的运行速度。但是,如果在每一个模块的后面都加入寄存器的话,会用到非常多的寄存器资源,这一方面会受到 Zynq 平台的资源限制,另一方面,Conv1+Conv2+Pool1 组成的流水线相比于 Conv1+Block RAM+Conv2+Block RAM+Pool1 组成的流水线在速度上并没有降低,属于无效的“面积换性能”。在 CNN 的层间合理的加入 Block RAM,既避免了片上存储资源的浪费,也提高了 CNN 在嵌入式平台上的运行速度。

3.4.4 层间通信及计算协同

Zynq 与 CPU 在处理数据时的本质区别, Zynq 的数据处理与时钟的时序密切相关。在一个时钟上升沿到来之后, Zynq 会同时执行所有的代码, 这种并行处理方式是 Zynq 的核心优势。CPU 是顺序执行代码的, 即一条指令执行完成后才会执行下一条指令, CPU 的设计通常使用高级编程语言 (如 C/C++), 这些语言描述了顺序执行的逻辑。而 Zynq 的设计通常使用硬件描述语言 (如 Verilog 或 VHDL), 这些语言描述了硬件电路的行为, 同一模块内的代码并不是严格按照顺序执行。需要在特定时间点, 通过使能信号触发, 否则会产生时序混乱, 影响系统的正常运行。在卷积核还未读取数据或读取完成之前进行卷积计算, 都将产生错误的结果。如图 3-9 所示, 图 3-9 (a) 为正常进行卷积计算的情况, 如图 3-9 (b) 所示, 卷积核还未读取完全, 卷积计算已经开始, 导致最后一位特征值为原始状态 (高阻态), 故计算得到错误结果 “X”。故在编写代码时, 需要修改代码中有关信号间时序关系的部分, 确保所有信号在计算时都处于正确的状态。

特征值		卷积核						
1	0	2	×	0	1	1	=	8
0	0	3		1	0	0		
5	0	1		1	1	1		
(a) 正确的卷积计算								
1	0	2	×	0	1	1	=	X
0	0	3		1	0	0		
5	0	X		1	1	1		
(b) 错误的卷积计算								

图 3-9 正确与错误的卷积计算情况对比图

Fig.3-9 Comparison diagram of correct and incorrect convolution calculation cases

CNN 通常由多个计算层组成, 每一层的计算都需要依赖前一层的输出数据。为了实现模型的正确推演, 必须确保不同层之间的计算能够协同运作, 即前一层的计算完成后, 后一层的计算才能开始。en 信号是一个使能信号, 用于控制每一层的计算是否开始。如图 3-10 所示, 在第一卷积模块的计算结果送到第二卷积模块之后, 第二卷积模块开始计数, 当第二卷积层达到卷积计算的条件以后才能让使能信号处于有效状态, 此时, 第二卷积层才能开始进行卷积运算。池化模块在输出特征值的过程并不是连续的, 相邻特征值之间往往都会有至少 1 个时钟周期的不包含任何信号的低电平信号, 所以, 在把数据存储到 Block RAM 时, 需要池化模块输出特征值

的同时再生成一个同步的使能信号，用于避免存入无效的数据或生成错位的数据存储地址。只有当 en 信号有效时，该层的计算模块才会开始运行。在每一层的计算模块（如卷积计算模块、池化模块、全连接模块）中都加入了 en 信号，用于控制该层的计算。通过 en 信号的控制，确保每一层的计算在前一层计算完成后才开始，从而避免数据未准备好时进行计算导致的错误，通过协同运作，确保系统的各个模块能够按照正确的时序运行，这种设计方法不仅提高了系统的稳定性，还优化了硬件资源的利用，适合在 Zynq 平台上实现高性能的 CNN。

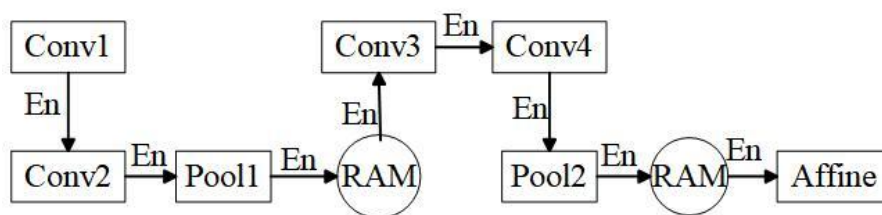


图 3-10 层间通信使能信号示意图

Fig.3-10 Diagram of inter-layer communication enable signal

总体而言，在硬件实现中，模块间通信设计简化了各模块的整合与编译过程，便于整体梳理和检错，并避免了数据误存储和误读取的情况。这种设计方法不仅提高了系统的性能和稳定性，还简化了硬件实现的流程，适合在 Zynq 平台上实现高性能的 CNN。

3.5 实验结果与分析

本文所搭建的系统在 Xilinx Zynq-7020 平台上得以实现，把 CNN 各模块都部署在 PL 端，占用的资源如表 3-2 所示。

表 3-2 Zynq 资源占用情况

Tab.3-2 Zynq resource utilization

Resource	Utilization	Available	Utilization%
LUT	28875	53200	54.28
LUTRAM	238	17400	1.37
FF	15452	106400	14.52
BRAM	78	140	55.71
DSP	220	220	100.00
IO	69	125	55.20

如表 3-3 所示,文献[61]至文献[65]中提出的都是基于 LeNet-5 改进的轻量化 CNN 模型,其中文献[61]与文献[62]都是只使用一个全连接层,以及略微缩减各个卷积层的通道数,两篇文献中的轻量化模型的区别主要体现在第二卷积层通道数的不同。文献[63]把各卷积层的通道数改为最小,其第一、二卷积层分别只设置 2、4 个卷积通道,依此达到模型轻量化的目的。文献[64]则是把卷积层、池化层和全连接层都改为一层,整体模型使用的参数量也比较可观。文献[65]增加一层卷积层和池化层,减少一层全连接层,且在第三池化层使用全局最大池化的方式对特征值进行展开。

与文献[61]至文献[65]中的轻量化 CNN 模型相比,本章搭建的 CNN 模型减少全连接层数和卷积通道数,使得参数量降低,再通过增加卷积层的数量来平衡卷积通道数的减少对网络特征提取能力的影响,使准确率仍保持在较高的水平,较少的参数量通常会减少模型的计算量,达到节省时间和计算资源的目的。

表 3-3 轻量级 CNN 模型与本设计的参数量对比

Tab.3-3 Comparison of parameter quantity between lightweight CNN models and this design

	文献[61]	文献[62]	文献[63]	文献[64]	文献[65]	本设计
卷积层个数	2	2	2	1	3	4
池化层个数	2	2	2	1	3	2
全连接层个数	1	1	2	1	1	1
卷积层参数	1950	60	2550	100	4500	1068
全连接层参数	1920	3110	2560	5760	176	1290
总参数	3870	3170	5110	5860	4676	2358
识别准确率	NA	98.12%	97.17%	96%	95%	98.20%

RTL 编码比 HLS 和 OpenCL 等高级综合平台有更好的硬件实现效率和严格的时序控制。利用卷积计算的并行性和流水线操作的方式,通过充分扩展卷积计算电路,获得更好的加速效果。

如图 3-11 所示,在 50 MHz 的工作频率下,从 139.17 μs 开始读取 28 $\times$ 28 的图像数据,到 225.19 μs 得出目标分类结果,完成对一次手写体数字识别只需要 86.02 μs ,全程仅仅经过了 4301 个时钟周期。如表 3-4 所示,与其他参考文献相比,本文在实时性能上相比 HLS 等平台上的实现有显著提升。

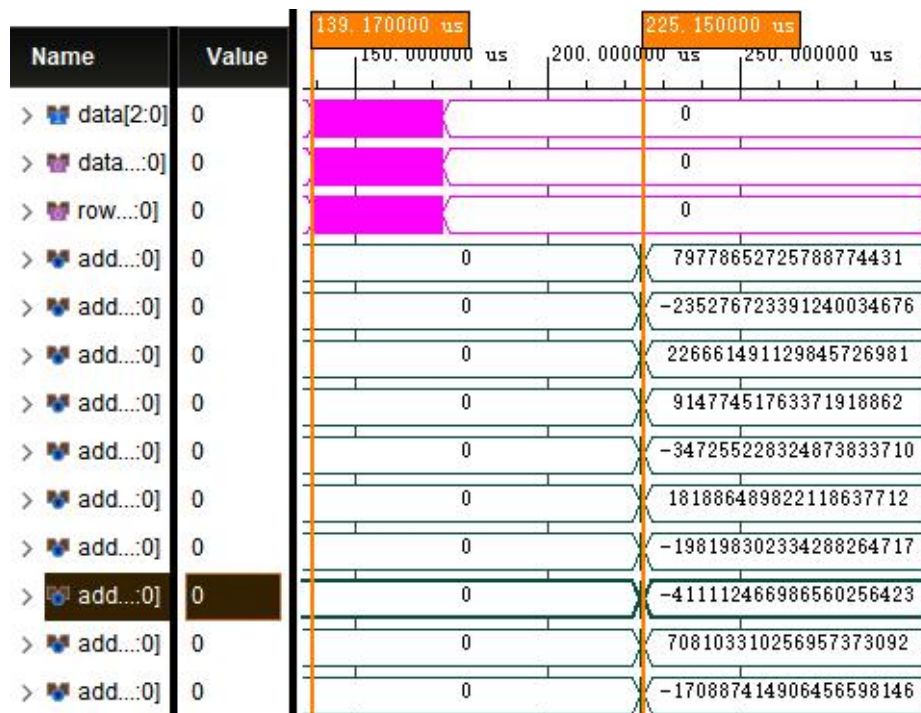


图 3-11 预测延迟仿真图

Fig.3-11 Simulation graph for prediction delay

表 3-4 手写数字识别速度对比

Tab.3-4 Comparison of handwritten digit recognition speed

	文献[66]	文献[67]	文献[68]	文献[69]	文献[70]	本设计
平台	Pynq -Z2	Zynq XCZU9EG	Zynq -7000	Zynq -7020	Zynq -7020	Zynq -7020
频率/MHz	136	100	166	200	50	50
识别时间/ μ s	960	4631	151	129.94	653	86.02
识别周期	130560	463100	25066	27988	32650	4301

3.6 本章小结

本章使用 Verilog 编程语言在 Zynq 上完成了对 CNN 模块的 RTL 编码，采用并行计算与流水线操作相结合的方式实现对 CNN 的加速，在 Xilinx Zynq-7020 平台完成对基于 CNN 的手写体数字识别系统的搭建，对 MNIST 数据集的手写体数字识别率达到 98.2%。本章搭建的轻量级 CNN 模型在降低卷积计算量的同时，保持了较高的识别准确率，采用 RTL 编码形式在 Zynq 上搭建的系统，在实时性方面有着显著的优势。

4 基于 Zynq 的头盔检测技术硬件平台设计

鉴于 YOLOv4 - tiny 是专为资源受限场景设计的，支持模型剪枝和量化等后处理技术，本章选择以该网络结构为基础，采用乒乓缓冲、通道剪枝和特征值量化等技术完成对 YOLOv4-tiny 模型的轻量化设计，使用 HLS 进行卷积和采样模块的硬件加速器设计与优化，将其部署在 Zynq-7020 开发板上，并完成对该网络模型的硬件加速。

4.1 YOLOv4-tiny

YOLOv4-tiny 是 YOLOv4 的轻量级版本，专为计算资源有限的场景设计，例如嵌入式设备、移动平台和实时应用。它在保持较高检测精度的同时，显著降低了计算复杂度和模型参数量，使其能够在资源受限的设备上高效运行。YOLOv4-tiny 继承了 YOLO 系列单阶段检测框架的优势，能够直接在单次前向传播中完成目标检测任务，具有速度快、效率高的特点。

YOLOv4-tiny 和 YOLOv4 是同一系列的目标检测模型，但它们在性能、速度和计算复杂度上有显著差异。从检测精度（mAP）、推理速度（FPS）、运算速度（GOPS）和参数量（Params）四个方面对两者进行的详细对比如表 4-1 所示。

表 4-1 参数对比

Tab.4-1 Parameter comparison

模型	检测精度（mAP）	推理速度	运算速度	参数量
YOLOv4-tiny	~21.7%	~200FPS	5-10GOPS	6-7MB
YOLOv4	~43.5%	~50FPS	100-200GOPS	60-70MB

如图 4-1 所示，YOLOv4-tiny 的网络结构主要由三部分组成：Backbone（骨干网络）、Neck（颈部网络）和 Head（检测头）。其 Backbone 采用了 CSPDarknet53 的轻量版。CSPDarknet53-tiny 在显著降低计算复杂度与参数量的同时，凭借跨阶段部分连接（Cross Stage Partial Connections, CSP）设计强化特征提取能力与多样性，从输入图像提取多层次特征，助力 YOLOv4-tiny 在资源受限设备上实现高精度检测。CSP 结构通过将特征图分成两部分并分别处理，最后再合并，从而在减少计算量的同时增强了特征的多样性。Neck 部分采用了简化版的路径聚合网络（Path Aggregation

Network, PANet)，用于增强特征融合。PANet 通过自上而下和自下而上的双向路径，将深层语义信息与浅层细节信息结合起来，从而提升模型对多尺度目标的检测能力，尤其是对小目标的检测效果。Head 部分则负责输出最终的检测结果，包括目标的边界框坐标、类别概率和置信度分数。

在特征提取与融合方面，YOLOv4-tiny 通过 CSPDarknet53-tiny 从输入图像中提取多层次的特征，并在不同层级进行特征融合。这种多尺度特征融合机制使得模型能够同时捕捉到目标的全局语义信息和局部细节信息，从而提升检测精度。

YOLOv4-tiny 的损失函数设计继承了 YOLOv4 的优秀特性，主要包括分类损失、定位损失和置信度损失三部分。分类损失使用交叉熵损失函数来计算类别预测的误差，定位损失则采用 CIOU 损失函数（Complete Intersection over Union）来计算边界框的位置误差。

YOLOv4-tiny 目标检测模型最值得研究的地方在于，该模型支持模型剪枝和量化等后处理技术，可以进一步压缩模型大小并提升推理速度，使其更适合部署在计算资源有限的设备上。

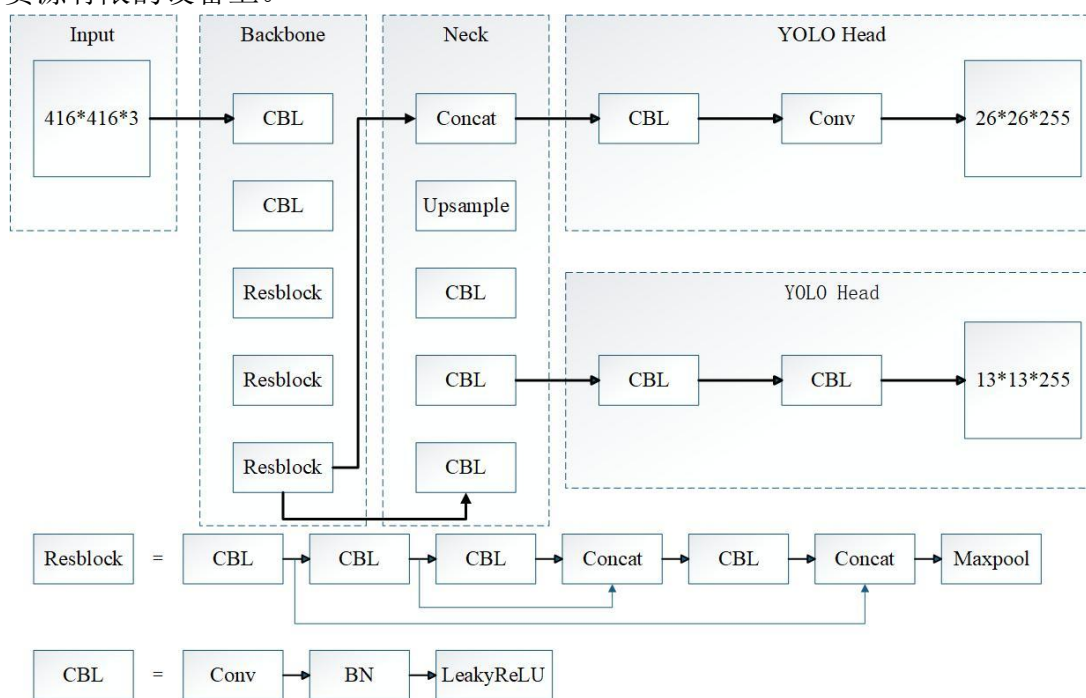


图 4-1 YOLOv4-tiny 目标检测模型图

Fig.4-1 Diagram of YOLOv4-tiny object detection model

4.2 乒乓缓冲策略设计

乒乓缓冲策略 (Ping-Pong Buffer Strategy) 是一种高效的双缓冲区数据交换技术, 其核心思想是通过两个缓冲区 (Buffer A 和 Buffer B) 的交替使用, 实现数据的无缝读取和写入, 从而避免数据处理过程中的等待和延迟。乒乓缓冲策略的优势在于其高效性和低延迟性。首先, 它能够实现数据的并行处理, 即在一个缓冲区进行数据写入的同时, 另一个缓冲区可以进行数据读取和处理, 从而最大化利用系统资源。其次, 由于两个缓冲区的角色切换是动态的, 系统无需等待数据完全处理完毕即可开始下一轮操作, 显著减少了等待时间。此外, 乒乓缓冲策略还具有较强的灵活性和可扩展性, 可以根据实际需求调整缓冲区的大小和数量, 以适应不同的应用场景。

如图 4-2 所示, 当完成对 “Read1” 部分的读取后, 利用嵌入式平台可以并行计算的特点, “Computer1” 部分在对 “Read1” 部分的数据进行乘加计算的同时, 继续对 “Read2” 部分的数据进行读取。依次类推, 在 “Computer1” 部分计算完成, “Save1” 部分开始对计算结果进行保存时, “Computer2” 部分也开始对 “Read2” 部分的数据进行乘加计算。时间 t_1 、 t_2 分别对应的是在乒乓缓冲、单缓冲模式下处理两组数据使用的时间。

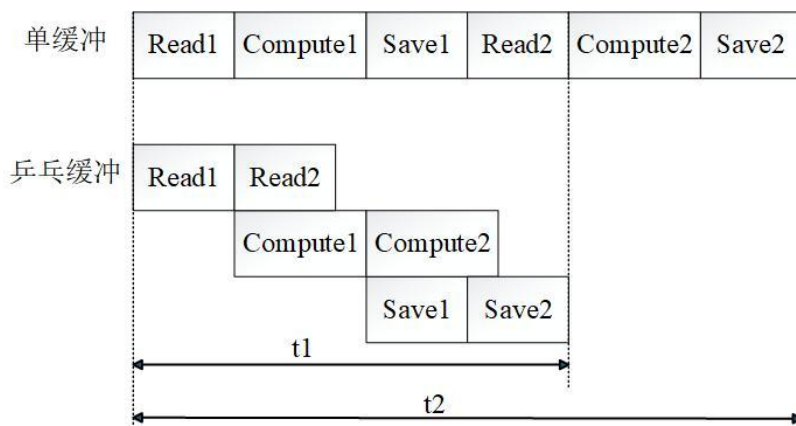


图 4-2 单缓冲与乒乓缓冲对比图

Fig.4-2 Single buffer versus ping-pang buffer

相比于按照 “Read-Computer-Save” 串行方式循环往复的单缓冲模式, 乒乓缓冲模式通过并行的、连续的数据处理模式, 很大程度地节省了数据在读取 (Read) 和保存 (Save) 过程上的时间, 在数据传输效率方面的提升是显著的。

4.3 算法的剪枝优化设计

4.3.1 模型稀疏化训练

稀疏化训练通过确认缩放因子（Scaling Factor）来实现通道剪枝（Channel Pruning），核心目标是通过引入稀疏性约束来减少模型中的冗余参数，从而在保持模型性能的同时降低计算复杂度和存储需求，进而提升 YOLOv4-tiny 目标检测模型的效率和性能。

在 CNN 中，稀疏训练可以应用于卷积层的缩放因子（通常与批归一化层中的可学习参数相关）。缩放因子是批归一化（BN）层中的一个可学习参数，用于对归一化后的特征进行缩放。通过稀疏训练，缩放因子可以被优化为接近零的值，从而指示对应的特征通道是冗余的，可以被剪枝。对卷积层进行稀疏化训练的具体公式如（4-1）和（4-2）所示：

$$N = \frac{N_{in} - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \quad (4-1)$$

其中， N_{in} 为输入特征值， μ_B 为输入特征值的平均值， σ_B^2 为输入特征值的标准差， ε 为很小的常数（通常为 10^{-5} ），用于防止除零错误。

$$N_{out} = \gamma N + \beta \quad (4-2)$$

其中， N_{out} 为输出特征值， γ 为缩放参数， N 为归一化后的值， β 为平移参数。

在 YOLOv4-tiny 模型中，卷积模块是特征提取的核心组件，其在生成特征图的过程中需要对输入数据进行卷积操作和 LeakyReLU 激活，其中的卷积操作是一种线性变换。缩放模块位于卷积模块之后，目的是调整特征图的幅度，一般是通过对输入数据的每个通道都乘以一个缩放因子来实现，这与卷积操作属于同一种线性变换。理论上，卷积模块的卷积操作与缩放模块的操作进行整合，例如，卷积模块中的权重参数先根据缩放因子进行缩放，再进行卷积操作，这样做相当于把缩放模块隐藏在了卷积模块中。

批归一化层对输入数据进行归一化（减去均值并除以标准差），使得每个通道的数据分布趋于稳定，如公式 4-2 所示，批归一化层引入两个可学习参数：1）缩放因子（ γ ）对归一化后的数据进行缩放；2）偏移因子（ β ）对归一化后的数据进行偏移。缩放模块可以放在批归一化层的前面，缺点是归一化过程会导致不同通道的缩放因子大小相等，让缩放模块失效。缩放模块也可以放在批归一化层的后面，缺点

是使得各通道的重要性识别变得复杂，每个通道都要进行两次连续的缩放因子，连续的缩放会导致评估结果产生变化。

本章提出，在进行稀疏化训练的过程中，批归一化层的缩放因子与缩放模块的缩放因子相结合，这样做不仅可以避免额外的计算开销，还能通过新的缩放因子的大小直接评估通道的重要性，达到提高模型的效率和简洁性的目的。

批归一化层的操作过程如图 4-3 所示，卷积模块有多个特征通道（如 C_{i1} 、 C_{i2} 、 C_{i3} 等），每个通道都对应一个通道缩放因子，它们在初始网络中都具有非零特性（如 C_{i1} 的缩放因子为 1.170， C_{i4} 的缩放因子为 0.003），意味着这些通道在初始状态下都是活跃的。稀疏化训练会使部分通道的缩放因子趋近于零，缩放因子越小，代表这些通道对模型的贡献较小；缩放因子越大，则表明这些通道对模型的贡献较大。移除缩放因子接近零的通道，剪枝后的网络只保留对模型贡献大的通道，从而生成一个更加精简高效的模型，既保持了模型性能，又显著降低模型的计算复杂度和存储需求。

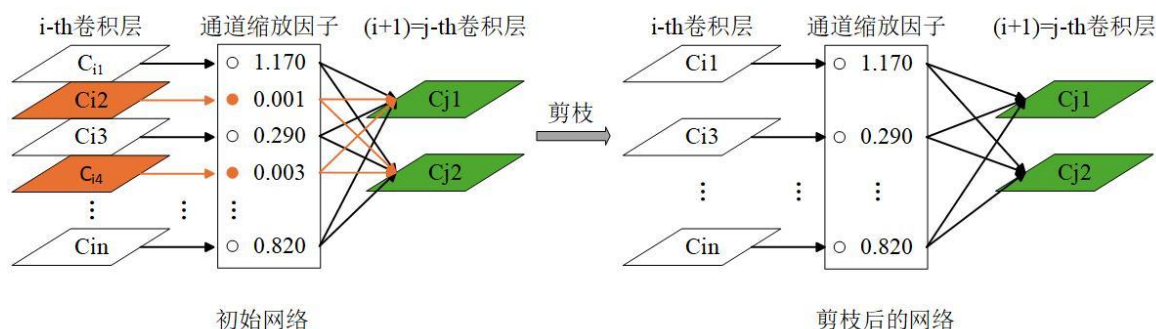


图 4-3 批归一化层运行原理图

Fig.4-3 Schematic diagram of batch normalization layer operation principle

4.3.2 通道剪枝

一次迭代并不能完成对初始网络的剪枝，通过一次又一次对缩放因子的训练，不断地把缩放因子接近零的通道剪枝掉，才能获得一个最佳的 YOLOv4-tiny 剪枝模型。所以，在通过批归一化层对卷积层的各通道进行剪枝后，需要利用预训练模型已经学习到的通用特征对剪枝后的网络进行调整，快速适应下一次的迭代任务。本章将按照“归一化训练-剪枝-调整”的步骤进行一次次迭代，从而获得对 YOLOv4-tiny 模型最佳的剪枝结果。

在对 YOLOv4-tiny 模型做通道剪枝的迭代过程中，需要设置一个全局阈值来判断模型中哪些特征通道是对模型贡献小的，是能够被剪枝的。通俗来说：通道缩放

因子如果比全局阈值小，则该通道对整个模型来说就是冗余的，也是可以被剪枝的。全局阈值通常被设置为所有通道缩放因子（如 BN 层中的 γ 参数）绝对值的第 n 百分位数。例如，如果设置为第 10 百分位数，则只有缩放因子绝对值最小的 10% 通道会被剪枝。全局阈值的选择十分重要，会直接影响剪枝的稀疏程度和模型的最终性能。

剪枝 mask 是通道剪枝中的另外一个关键工具，主要用于标记和移除不重要的权重或通道。通过生成剪枝 mask，可以显著减少模型的参数量和计算量，从而提高模型的效率和部署灵活性。剪枝 mask 的生成通常基于某种重要性评估标准，例如权重的绝对值、缩放因子（如 BN 层中的 γ 参数）或梯度信息。生成剪枝 mask 的步骤如下：

（1）根据预定义的标准（如权重的绝对值或缩放因子的值），计算每个权重或通道的重要性。

（2）将所有权重或通道按重要性排序，并根据设定的阈值（如全局阈值）筛选出需要剪枝的部分。

（3）对于重要性低于阈值的权重或通道，在 mask 中对应位置标记为 0（剪枝）；对于重要性高于阈值的权重或通道，标记为 1（保留）。

在 YOLOv4-tiny 模型中还有一个非线性变换层（shortcut 层），主要功能是将输入数据直接传递到后续层，缓解梯度消失问题，加速模型训练，并提高模型的泛化能力。不仅如此，shortcut 层还可以通过跳过某些复杂的计算过程，减少模型的参数量，提高模型的效率。shortcut 层通常与卷积层或池化层一起使用，卷积层和池化层是线性变换层，而 shortcut 层通过非线性变换增强了模型的表达能力。

本章设计的通道剪枝具体实现过程可以总结为以下三个步骤：

（1）全局阈值与 mask 生成：在通道剪枝中，首先使用全局阈值来确定每个卷积层的 mask。全局阈值通常基于缩放因子（如 BN 层中的 γ 参数）的绝对值，通过设定一个统一的阈值（如第 n 百分位数），筛选出需要剪枝的通道。

（2）shortcut 层的 mask 合并：将与 shortcut 层连接的所有卷积层的剪枝 mask 进行整合。这是因为 shortcut 层直接传递输入数据，其剪枝操作会影响多个卷积层的一致性。

（3）剪枝操作：使用合并后的 mask 对卷积核进行剪枝，同时限制每层中保留的通道数量。这种操作不仅减少了 YOLOv4-tiny 模型的参数量，还优化了该模型的

性能。

通道剪枝的结果如图 4-5 所示，在迭代初期（0-100 次），mAP（平均精度）、Precision（精确率）以及 Recall（召回率）都会有一定程度的下降，这是因为剪枝操作移除了部分通道，导致模型的特征提取能力暂时减弱，也对部分目标的检测能力降低。迭代中期（100-200 次），mAP、Precision 和 Recall 都开始趋于稳定，这意味着：随着迭代次数的增加，YOLOv4-tiny 模型逐渐适应了剪枝后的结构，性能开始恢复，检测的准确性和对目标的覆盖率均逐渐提高。在迭代后期（200-300 次），mAP 可能进一步优化，表明剪枝后的 YOLOv4-tiny 模型在保持较高精度的同时，参数量和计算量显著减少。

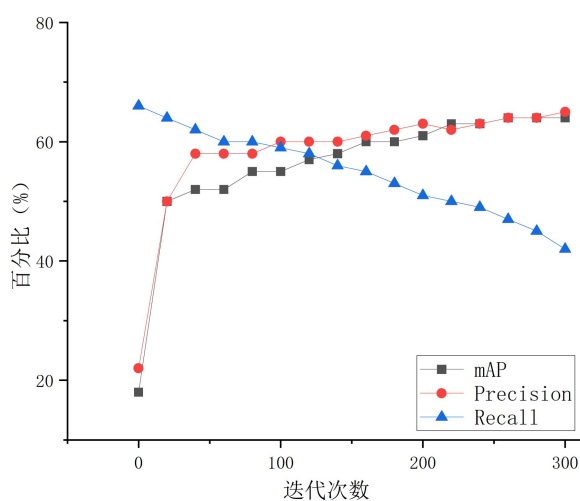


图 4-5 300 次通道剪枝迭代结果对比图

Fig.4-5 Comparison diagram of results from 300 iterations of channel pruning

通过对 YOLOv4-tiny 模型进行 300 次通道剪枝迭代的实验结果分析，在剪枝初期，模型的 mAP、Precision 和 Recall 可能会有所下降，这是由于剪枝操作移除了部分通道，导致模型的特征提取能力暂时减弱。通过微调训练，模型逐渐适应了剪枝后的结构，性能开始恢复并趋于稳定。随着迭代次数的增加，剪枝比例逐渐提高，表明模型在保持性能的同时，逐步移除了更多的冗余通道。

200-300 次迭代是模型性能优化的关键阶段，进一步迭代可能会导致性能下降。这种通道剪枝方法能够在减少模型复杂度的同时，保持较高的性能，非常适合部署在资源受限的设备上。

4.4 特征参数数据量化设计

量化是将高精度数据（如 32 浮点型数据）映射到低精度数据（如 8 或 16 位定点型数据）的过程。量化函数本质上是一个阶跃函数，其输出被离散化为有限的常数值。通过量化，可以减少模型的存储需求和计算复杂度，从而加速推理过程并降低能耗。这对于资源受限的设备（如移动设备或嵌入式系统）尤为重要。

本章计划把 32 位浮点型数据量化为 8 位定点型数据。把大精度范围的 32 位浮点型数据转换成仅 256 个不同值的 8 位定点型数据，这会导致 YOLOv4-tiny 模型性能下降。TensorRT 是 NVIDIA 开发的一个高性能深度学习推理库，通过一系列优化技术（如层融合、精度校准、动态张量内存管理等）显著提高模型的推理速度，并减少内存占用，特别适合部署在实时性要求高、资源受限的环境中。

TensorRT 支持多种精度模式的推理，包括 32 位浮点型、16 位浮点型和 8 位定点型。其中，8 位定点型数据量化是 TensorRT 的重要特性之一。首先，减少模型的计算量和内存占用，显著加速推理过程，通过校准工具，TensorRT 能够以最优的方式将 32 位浮点型数据转换为 8 位定点型数据精度，最小化性能损失。然后，使用校准数据集生成量化参数，确定每个层的动态范围。最后，在推理过程中，将 32 位浮点型数据量化为 8 位定点型数据，并在计算完成后反量化为 32 位浮点型数据，形成量化与反量化之间的闭环。

如图 4-8 所示，量化训练的目标是通过引入量化操作，使模型在低精度（如 8 位定点型数据）下仍能保持较高的性能。训练过程包括以下几个步骤：

（1）在训练开始时，模型的权重被初始化。这些权重通常是 32 位浮点型的数据，用于后续的浮点推理和量化操作。

（2）在每次训练迭代中，模型的权重被载入并更新，训练迭代的过程中不断调整载入的权重参数，利用反向传播算法完成对权重参数的更新。

（3）使用高精度的 32 位浮点型权重进行浮点推理，计算模型的输出，浮点推理的主要作用是获取权重数据的动态范围，为后续的量化提供参考。

（4）在浮点推理的基础上，进行定点推理。定点推理使用低精度的 8 位定点型权重进行计算，以减少计算量和内存占用。

（5）将高精度的 32 位浮点型权重量化为低精度的 8 位定点型权重，量化过程通过将浮点数据映射到离散的整数值来实现。

（6）在定点推理后，将低精度的 8 位定点型数据反量化为高精度的 32 位浮点

型数据，以便进行后续的计算和损失函数的评估。

(7) 根据模型的输出和真实标签计算损失函数，并通过反向传播算法更新模型的权重。

(8) 检查训练是否达到停止条件（如达到迭代次数或损失函数收敛）。如果满足停止条件，则保存训练结果；否则，继续下一轮训练。

前向计算是模型推理的核心部分，通过加载权重和数据，逐层计算模型的输出。前向计算过程包括以下几个步骤：

(1) 加载输入图像数据，这些数据通常是高精度（如 32 位浮点型）的。

(2) 加载当前层的权重，这些权重可能是高精度（32 位浮点型）或低精度（如 8 位定点型）的，具体取决于量化设置。

(3) 使用低精度（如 8 位定点型）权重进行当前层的定点推理，定点推理通过量化操作减少计算量和内存占用。

(4) 检查当前层是否是最后一层，如果是最后一层，则输出结果；否则，继续下一层的计算。

(5) 输出模型的最终推理结果，这些结果通常是高精度（32 位浮点型）的。

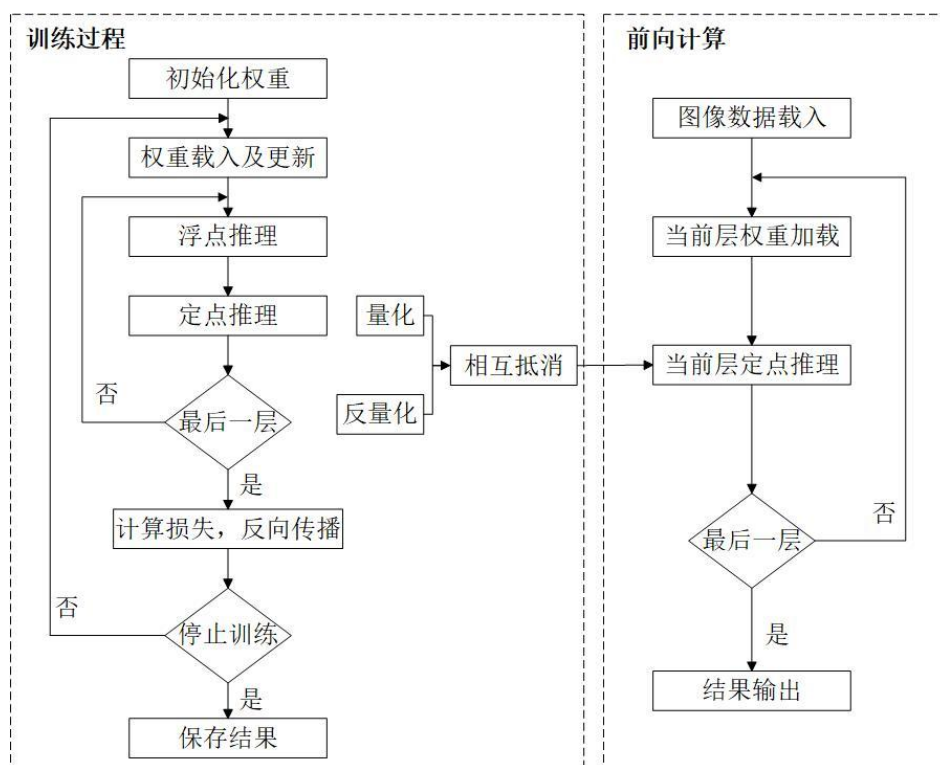


图 4-8 量化训练过程与前向计算流程图

Fig.4-8 Flowchart of quantization training process and forward computation

量化训练和前向计算的结合，能够在减少模型计算量和内存占用的同时，保持较高的推理精度，非常适合部署在资源受限的设备上。

4.5 高层次综合工具实现模块加速设计

HLS 是一种将高级编程语言（如 C、C++ 或 SystemC）描述的算法自动转换为硬件描述语言（如 VHDL 或 Verilog）的技术。与传统硬件设计方法相比，HLS 允许设计者在更高的抽象层次上进行工作，从而显著提高设计效率。通过 HLS，设计者可以专注于实现算法逻辑，而无需深入底层硬件的细节。HLS 工具能够自动处理复杂的硬件设计任务，如时序优化、资源分配和并行化，从而缩短开发周期并降低设计难度。此外，HLS 支持快速迭代和验证，使得在 Zynq 平台上实现复杂算法（如 YOLOv4-tiny 网络模型）更加高效和灵活。

在基于 Zynq 的 YOLOv4-tiny 网络模型开发中，HLS 技术发挥了关键作用。作为一种轻量级目标检测模型，YOLOv4-tiny 的计算密集型操作（如卷积和池化）可以通过 HLS 高效实现。通过 HLS，设计者可以用高级语言描述模型的计算模块，并通过工具自动生成优化的硬件电路。这不仅加速了开发过程，还能充分利用 Zynq 的并行计算能力，提高模型的推理速度。此外，HLS 支持模块化设计，便于对 YOLOv4-tiny 的各个子模块进行独立优化和验证，从而在性能和资源使用之间实现更好的平衡。

4.5.1 卷积模块加速设计

池化操作是 CNN 中的重要步骤。池化层对单个输入特征图进行采样，执行平均池化和最大池化，其主要功能是降维和提取主要特征。与卷积操作使用加法器和乘法器作为计算单元不同，池化操作主要使用比较器。与卷积类似，池化也是通过滑动窗口的方式进行的。池化核心从单个输入缓冲区读取所有数据，并将其与当前最大值进行比较，比较后将新的最大值写入输出缓冲区。

YOLOv4-tiny 模型的输入图像大小是 416×416 ，精度为 8 位定点型。通过并行工作方式与加法树与乘法树的流水线方式相结合，完成卷积模块的加速设计，提高该模块的并行性，减少延迟。池化层的 IP 核设计并未单独进行，而是直接替换为卷积层 IP 核中的比较操作。

本节使用的是 Vivado 2017.4 版本，利用 HLS 工具把卷积模块封装成 IP 核的步

骤如下：

- (1) 完成对卷积模块算法的头文件（.h）和源文件（.cpp）的编写。
- (2) 定义 IP 核的输入/输出数据类型。
- (3) 在 Block Design 上调用 IP 核，并设置总线协议 AXI。
- (4) 通过仿真完成对模块的验证。

如图 4-9 所示为卷积模块的 IP 核结构，本章通过模块化设计和接口优化，确保了硬件模块的高效运行和灵活性。HLS 设计流程包括软件程序编写和仿真封装，涵盖了从算法实现到功能验证的完整过程。如果布局和布线优化不当，IP 核的连接可能会变得复杂或错误，导致无法生成比特流文件。所以，在设计硬件 IP 核时，必须考虑硬件电路的布线优化，以确保生成的比特流文件能够正确运行。这种设计方法不仅降低了计算复杂度，还提高了硬件模块的可维护性和可扩展性。

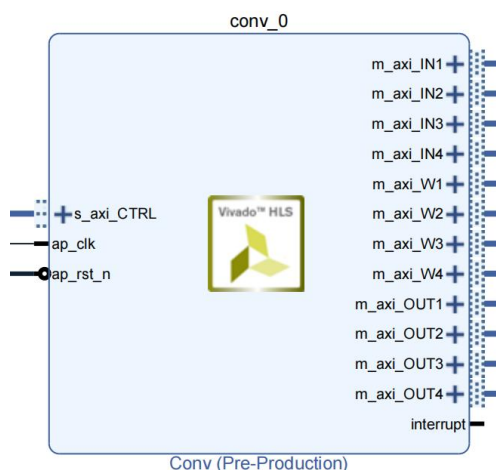


图 4-9 卷积模块 IP 核架构图

Fig.4-9 Architecture diagram of convolution module IP core

4.5.2 上采样模块加速设计

在 YOLOv4-tiny 目标检测网络中，上采样模块在生成两个检测头的过程中起到了至关重要的作用。检测头是该网络模型的核心部分，负责生成不同尺度的特征图，以检测不同大小的目标。上采样模块通过将低分辨率的特征图（如 13×13 ）转换为高分辨率的特征图（如 26×26 ），增强了特征图的信息表示能力，从而提高了检测头的性能。在众多上采样方法中，本文选择使用最近邻插值（Nearest Neighbor Interpolation），将每个像素的值复制到你邻近的多个位置，该方法简单，且计算量

较小。

如图 4-10 所示，以尺寸为 2×2 的输入特征图为例，包含 1、3、5、7 四个特征值。输出是尺寸为 4×4 的特征图，按照输入特征图四个特征值的位置，将输出特征图均等地分为 4 等份，每一份都包含四个相同的与输入特征值相对应的特征值。

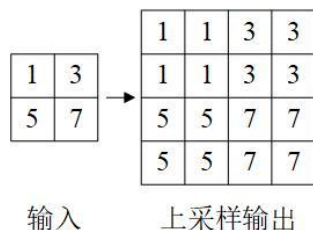


图 4-10 上采样最邻近插值流程

Fig.4-10 Nearest neighbor interpolation process for upsampling

由此可以推导出以下输入和输出数据地址的公式（4-3）：

$$new_addr = (old_addr \div size) \times size + (old_addr // size) \times 2 \quad (4-3)$$

其中 old_addr 是上采样前数据的地址， new_addr 是上采样后数据的地址， $size$ 是上采样前特征图的大小。 $\div$ 运算符计算除法的整数部分， $//$ 执行取模操作。

因此，上采样数据的地址可以直接通过原始数据地址和当前层的大小确定。在整个网络中，这种固定大小的上采样操作仅发生在最终特征提取时，这意味着该模块不具备复用特性。

考虑到上述因素，决定将计算任务交给 PS（处理系统）。首先，PS 通过 AXI 总线从 DDR（动态随机存取存储器）中读取输入特征图数据，输入特征图是 13×13 的低分辨率，存储在 DDR 的特定区域。然后，PS 运行上采样算法（最近邻插值），将低分辨率特征图转换为高分辨率特征图，尺寸为 26×26 ，地址转换如公式 4-3 所示。最后，PS 将上采样后的高分辨率特征图数据写入 DDR 的输出特征图区域。输出特征图通常存储在 DDR 的另一个特定区域，以便后续处理模块（如检测头）使用。

利用 PS 设计上采样模块是一种高效且灵活的方法，特别适合处理计算简单但数据量较大的任务。这种设计方法不仅充分利用了 PS 的灵活性和易用性，还释放了 PL 资源，用于处理更复杂的计算任务。上采样模块 IP 核如图 4-11 所示。

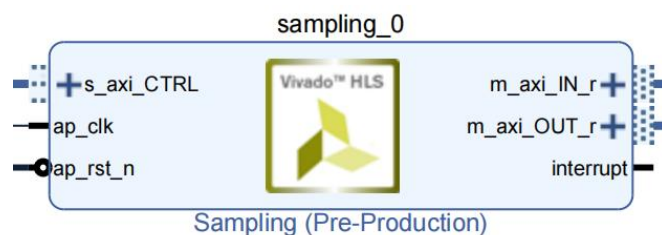


图 4-11 上采样模块 IP 核架构图

Fig.4-11 Architecture diagram of upsampling module IP core

4.6 实验结果分析

4.6.1 实验环境

在硬件设计方面,选择 Zynq-7020 平台,该平台搭载了 Xilinx XC7Z020-2CLG400I 芯片,包含的资源如表 4-2 所示。

表 4-2 Zynq-7020 开发板硬件资源表

Tab.4-2 Hardware resource table for Zynq-7020 development board

资源类别	具体数值
FPGA 型号	XC7Z020-2CLG400I
内核 ARM	双核 ARM Cortex-A9
内存	DDR3, 1GB; 数据速率 1066Mbps
逻辑单元数量	85K
查找表	53200
触发器	106400
乘法器	220
BLOCK RAM	4.9Mb
PS MIO	8 个
PL IO	94 个

开发工具选择了 Xilinx 公司的 Vivado 2017.4 作为硬件平台开发和仿真软件。Vivado 是 Xilinx 公司提供的一款集成开发环境 (IDE), 支持 Zynq 的硬件设计、仿真和调试。使用 Verilog HDL 语言进行 RTL 级 (Register Transfer Level) 硬件电路设计。RTL 级设计是硬件设计的关键阶段, 用于描述硬件电路的数据流和控制逻辑。硬件电路的时钟频率设置为 100 MHz。

验证过程从软件优化开始，逐步过渡到硬件实现和性能验证，具体步骤如下：

(1) 在 Pycharm 中对神经网络进行优化，获取最终的神经网络结构和相关权重参数，Pycharm 是一个常用的 Python 集成开发环境，适合深度学习模型的开发和调试。使用测试样本图像验证网络的性能，确保网络在软件端的输出符合预期。

(2) 将 Pycharm 中网络各层的结果与 Vivado 中计算的结果进行比较，以确保输出一致。这一步骤是硬件验证的关键，确保硬件实现与软件模型的一致性。

(3) 通过 Vivado 进行前端仿真，确认硬件电路的功能正确性。前端仿真主要用于验证硬件设计的逻辑功能。对每个模块的电路进行静态时序分析（Static Timing Analysis, STA），以避免亚稳态问题。静态时序分析是硬件设计中的重要步骤，用于确保电路在指定时钟频率下能够稳定运行。

(4) 将 Verilog HDL 程序下载到 Zynq 开发板中，进行硬件实现，在开发板上运行硬件电路，验证其实际检测性能。这一步骤是硬件开发的最终目标，确保硬件电路在实际应用中能够满足性能要求。

4.6.2 平台搭建与上板验证

完成卷积和上采样 IP 核的设计后，硬件路径结构图如图 4-13 所示。完成 Block Design 后，进行综合和编译，随后生成比特流文件。使用 JTAG 线将开发板连接到计算机，并将设计的项目下载到开发板中，如图 4-12 所示。

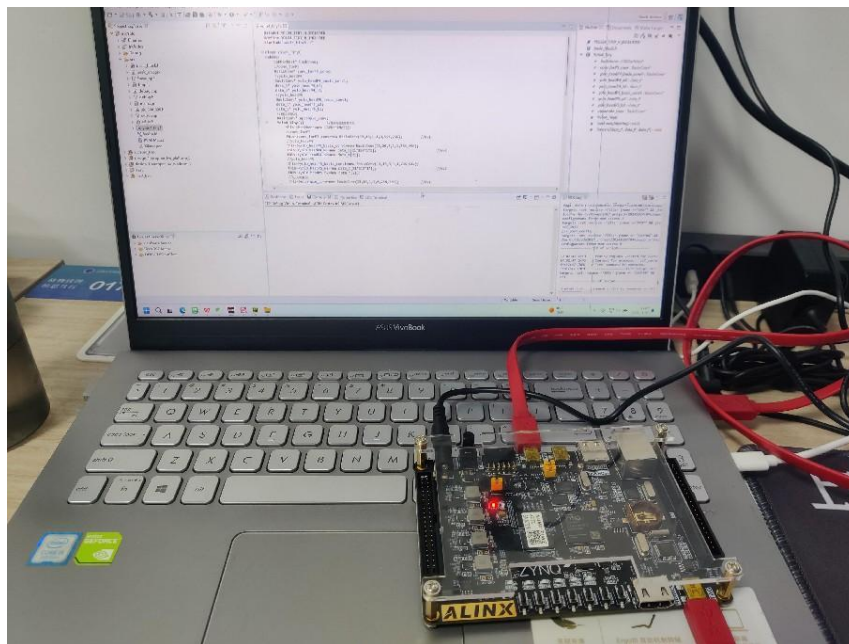


图 4-12 实验场景图

Fig.4-12 Experimental scenario diagram

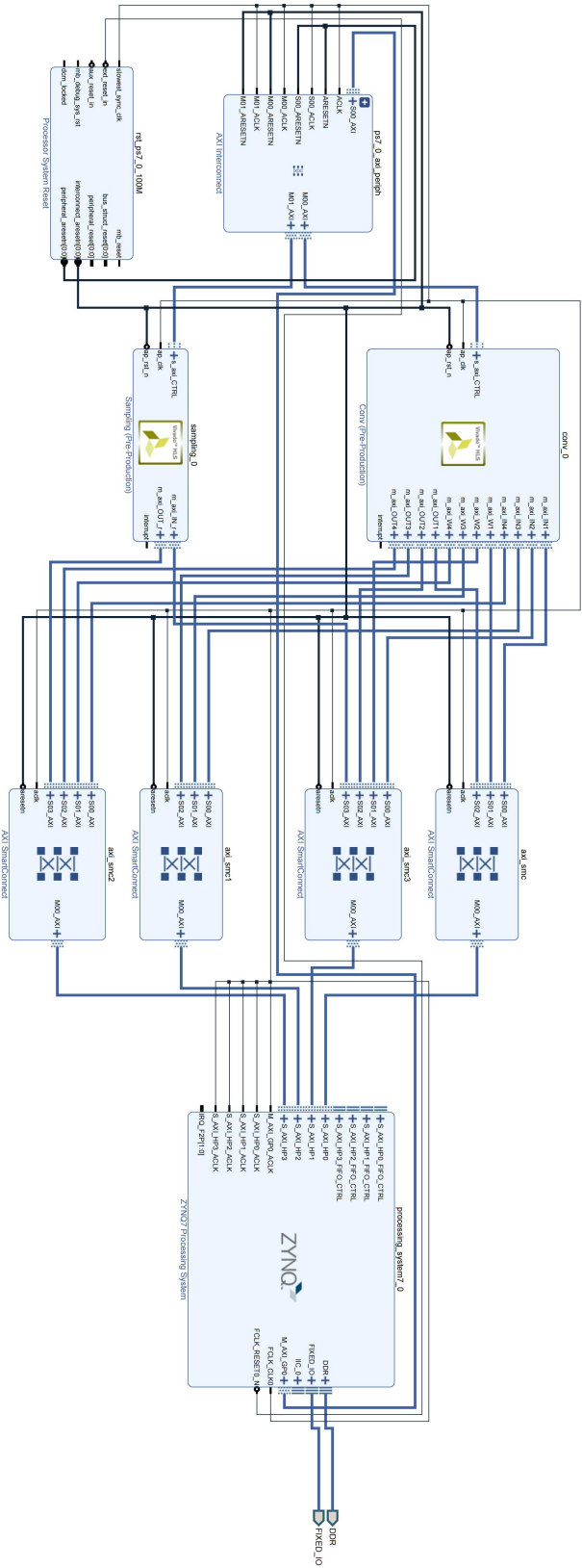


图 4-13 硬件路径结构图

Fig.4-13 Hardware path structure diagram

4.6.3 结果分析

如图 4-14 所示，先利用 Pycharm 软件对待检测图片进行预处理，把图片尺寸改为 416×416 ，把 jpg（或 png）格式的图片转换成 bin 文件，连同转换为 bin 文件的权重和偏置参数一同存入到 SD 卡中。把 SD 卡插入开发板，打开 Zynq-7020 开发板电源，如图 4-13 所示为目标工程烧录进开发板的场景。运行开发板，目标检测的结果以 bin 文件的形式存储在 SD 卡中，关闭电源，拔出 SD 卡，把得到的 bin 文件转换成 png 格式的图片，如图 4-15 所示为本章搭建的 YOLOv4-tiny 目标检测模型的检测结果展示。

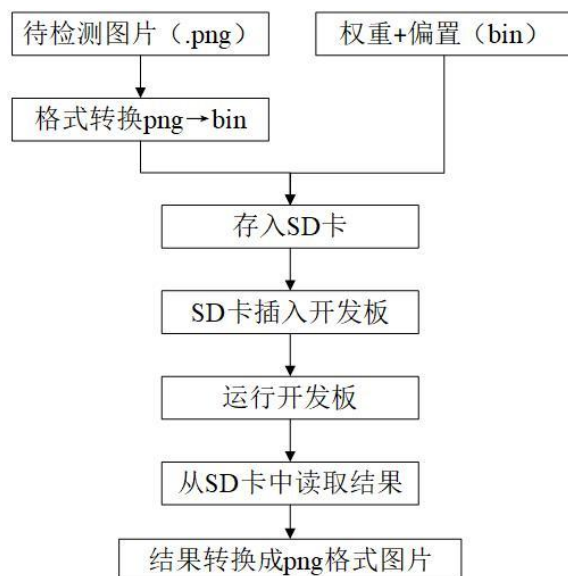


图 4-14 操作流程

Fig.4-14 Operational procedure



(a)待检测图片

(b)检测后图片

图 4-15 检测结果图

Fig.4-15 Diagram of detection results

图 4-16 展示了目标检测系统运行的资源使用情况和消耗比例。可以得出结论，

Zynq-7020 开发板上的资源足以满足目标检测系统的运行条件。Block RAM 仅使用了总存储空间的 54%，而 DSP 的使用量占总量的 51%。

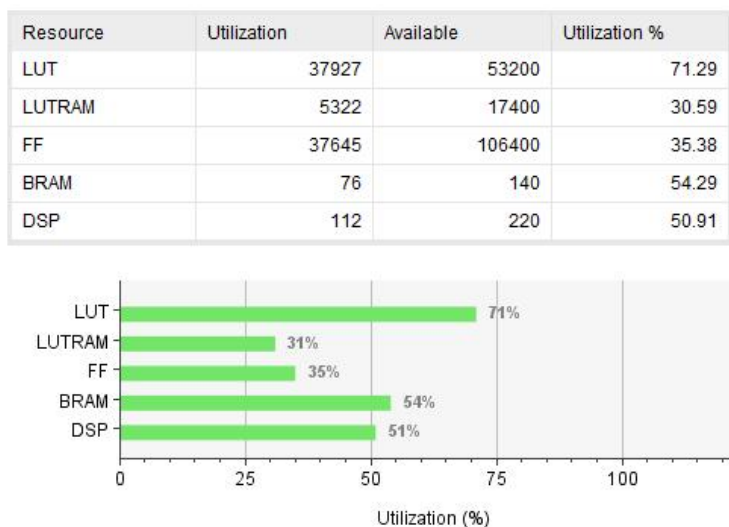


图 4-16 资源消耗与占用情况

Fig.4-16 Resource consumption and utilization

图 4-17 展示了系统的功耗为 2.483W，表明功耗较低且芯片工作温度合理，满足长期运行要求。

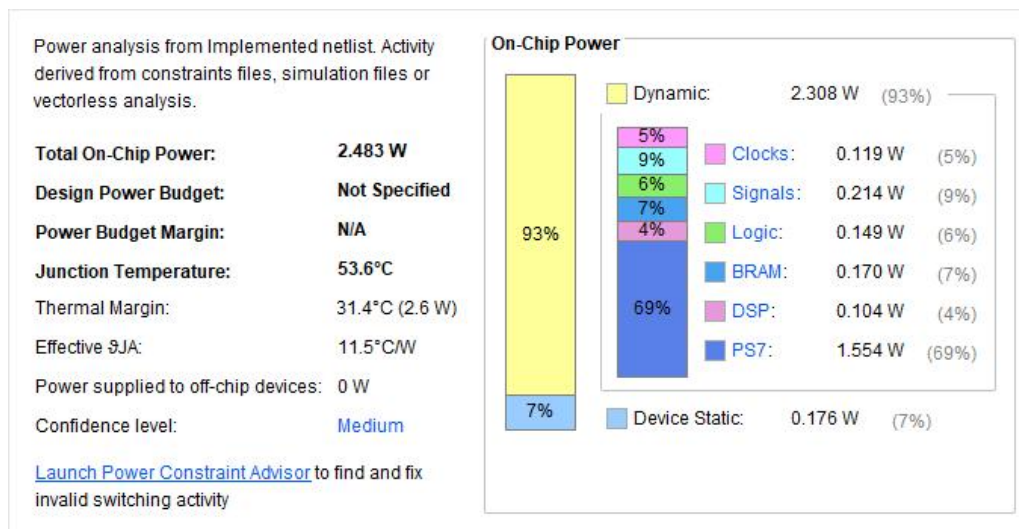


图 4-17 系统功耗图

Fig.4-17 System power consumption diagram

论文在 Zynq 平台上实现了整个目标检测系统的部署，如表 4-3 所示。与 CPU 平台相比，本文的目标检测系统在计算时间几乎相同的情况下，实现了 CPU 平台约 1/40 的功耗。此外，与纯嵌入式 CPU ARM 相比，这种实现方式提供了更快的检测

速度，检测速度约为纯嵌入式 CPU ARM 的 8 倍。该目标检测算法基于 CPU 的检测精度为 68.61%，其基于 Zynq 的检测精度为 65.42%，检测效果下降了 3.19%，下降程度在可以接受的范围内。为综上所述，在嵌入式平台上设计的目标检测系统满足了低功耗、低延迟的要求，实现了良好的加速性能。

表 4-3 多种平台指标对比

Tab.4-3 Comparison of multiple platform metrics

	CPU	CPU	Zynq
平台类型	Intel Core i5 9600	ARM-A9	XC7Z020
计算耗时/s	1.17	13.27	1.62
功耗/W	96	1.94	2.48
mAP	68.61%	-	75.42%

如表 4-4 所示，文献[71]利用 Vivado HLS 工具完成对所有模块的加速，采用 int16 定点量化，并在 Zynq-7000 系列的 XC7Z035 硬件平台上。其搭建的 YOLOv2-tiny 目标检测模型对开发板资源的利用率不高，未能充分体现嵌入式开发平台的优势。文献[72]在 Zynq-7020 平台上实现了 Tiny-YOLO 网络的部署，充分利用了 Zynq PL 端的各种资源，但其特征图划分方法缺乏通用性，难以直接应用于其他目标检测网络，模型实现的效率较低。文献[73]在 Zynq-7030 平台上实现了 YOLOv4-tiny 网络的部署，提供了特征图处理的通用方法并满足低功耗要求，但使用了较多的资源，增加了对嵌入式设备的性能要求。本章在确保低功耗的基础上，减少了硬件资源占用，降低了对嵌入式设备的性能需求，提升了模型的实际应用价值。

表 4-4 硬件平台工作对比

Tab.4-4 Comparison of hardware platform performance

	文献[71]	文献[72]	文献[73]	本文
模型	YOLOv2-tiny	Tiny-YOLO	YOLOv4-tiny	YOLOv4-tiny
平台	Zynq-7035	Zynq-7020	Zynq-7030	Zynq-7020
量化	int16	int8	int16	int8
LUT	43k	35k	34k	38k
FF	19k	14k	13k	38k
DSP	161	218	288	112
BRAM/Mb	8.42	4.82	8.71	2.74
功耗/W	2.51	2.72	2.69	2.48

4.7 本章小结

本章设计与实现了一个基于 Xilinx Zynq 平台的 YOLOv4-tiny 目标检测系统，重点介绍了系统的优化策略（如乒乓缓存策略和通道剪枝优化设计）、硬件加速（使用 HLS 工具对卷积模块和上采样模块进行硬件加速）以及最终的资源与功耗评估结果。通过 Vivado 工具对 Zynq 的资源利用情况进行评估，结果显示资源利用合理，未出现资源瓶颈。将最终程序下载到开发板中，由 PL 和 PS 协同完成目标检测任务，并与传统 CPU 实现方案相比，该方案的功耗显著降低，适合嵌入式场景。因此，该方案通过硬件加速和优化策略，显著提升了目标检测系统的性能和效率。

5 结论与展望

近年来, CNN 在模型架构、训练方法、应用场景和硬件加速等方面取得了显著进展。从经典的 AlexNet 到轻量化的 MobileNet, 从图像分类到目标检测和语义分割, CNN 已经成为计算机视觉领域的核心技术。未来, 随着硬件技术的进步和算法研究的深入, CNN 将在更多领域发挥重要作用。目标检测算法在嵌入式设备上应用的主要挑战包括高功耗、大参数量、高计算量等方面。通过模型轻量化和算法加速, 本文旨在解决目标检测算法在嵌入式设备上的应用难题, 推动其在低功耗平台上的发展, 具体工作如下:

(1) CNN 背景分析: 本文首先分析了 CNN 的发展历程, 重点研究了 LeNet-5 和 YOLO 等经典目标检测模型。随后深入探讨了基于 CNN 的目标检测算法的发展方向以及如何在硬件平台上实现对 CNN 的加速。最后, 对 CNN 相关算法的发展历程以及现有技术进行了分析, 在理论上, 为论文后续的设计与实现奠定了基础。

(2) CNN 模型模块分析: 本文分析了 CNN 模型的各个模块, 重点研究了各模块的算法实现原理。以 LeNet-5 为例, 描述了如何利用 Zynq 平台的特点改进 CNN 模型, 最终得到一个更适合部署在 Zynq 平台上的目标检测系统。这种方法充分利用了嵌入式硬件平台的低功耗、实时性等特点。

(3) LeNet-5 模型改进: 基于 LeNet-5 CNN 模型, 开发了一种新的、更轻量化的 CNN 模型, 更适合部署在 Zynq 平台上。与 LeNet-5 相比, 该模型在保证精度损失在可接受范围内的同时, 显著减少了 CNN 的计算量和参数量。本文提出了 CNN 在 Zynq 平台上的部署方案, 详细介绍了 CNN 各基础模块的 RTL 实现以及各模块间的通信和流水线设计方案及其实现。此外, 还在 Zynq 平台上实现了对 LeNet-5 模型权重的定点量化, 并确保精度损失在可接受范围内。该模型在搭载 Xilinx XC7Z020-2CLG400I 芯片的开发板上实现, 实验结果表明: 在 50 MHz 时钟频率下, 完成一张图片的预测用时为 86.02 μ s, 验证了其在实时性方面的优势。这项作为后续 Zynq 平台上复杂深度学习网络的实现奠定基础。

(4) YOLOv4-tiny 目标检测模型设计与实现: 本文对 YOLOv4-tiny 目标检测模型框架进行了基本分析。以 YOLOv4-tiny 为框架, 通过乒乓缓存策略、稀疏化训练和通道剪枝等技术手段对模型进行轻量化优化, 使其更适合部署在嵌入式平台上。

在确定最终的网络模型后，使用高层次综合工具加速卷积和上采样模块。基于 Zynq-7020 平台完成了目标检测系统的整体框架设计，并将具体任务分配给 PS 和 PL 部分。硬件实现包括批归一化层与缩放层的融合以及浮点型转定点型数据的量化设计，降低了目标检测模型的复杂性和硬件资源的消耗。最后，在 Xilinx 旗下的型号为 XC7Z020-2CLG400I 的开发板上完成了部署。该系统成功在嵌入式设备上实现了目标检测，检测结果分析显示：在检测时间为 1.62 s 的情况下，功耗仅为 2.48 W。这完成了基于 Zynq 的 CNN 加速器的设计与实现。

尽管本研究在 CNN 模型轻量化和嵌入式部署方面取得了一定成果，但在算法和硬件加速方面仍存在不足之处：在算法层面，新兴的神经网络架构正逐渐展现出超越传统 CNN 的性能优势，未来可研究这些新型架构在嵌入式平台的适配方案，探索混合架构在资源受限环境下的可行性。在硬件加速方面，随着 FPGA 工艺节点的不不断提升，可研究利用新一代可编程逻辑器件实现更复杂的神经网络加速。特别是 3D 堆叠存储技术与计算架构的协同优化，有望突破现有存储墙限制。

参考文献

- [1]McCulloch W S, Pitts W. A logical calculus of the ideas immanent in nervous activity[J]. The Bulletin of Mathematical Biophysics, 1943, 5: 115-133.
- [2]LeCun Y, Bengio Y, Hinton G. Deep learning[J]. Nature, 2015, 521(7553): 436-444.
- [3]Sarker I H. Deep learning: a comprehensive overview on techniques, taxonomy, applications and research directions[J]. SN Computer Science, 2021, 2(6): 1-20.
- [4]Zou J, Han Y, So S S. Overview of artificial neural networks[J]. Artificial Neural Networks: Methods and Applications, 2009: 14-22.
- [5]杨晓艳,邓淼磊,张德贤,等.基于判别模型的年龄不变人脸识别方法综述[J].计算机工程与应用,2023,59(24):16-25.
- [6]Lin Q, Wang R, Wang Y, et al. Target detection algorithm incorporating visual expansion mechanism and path syndication[J]. IEEE Access, 2023, 11: 56973-56982.
- [7]Liu Y, Jiang X, Yu X, et al. A wearable system for sign language recognition enabled by a convolutional neural network[J]. Nano Energy, 2023, 116: 108767-108776.
- [8]Luqman H, ELALFY E. Utilizing motion and spatial features for sign language gesture recognition using cascaded CNN and LSTM models[J]. Turkish Journal of Electrical Engineering and Computer Sciences, 2022, 30(7): 2508-2525.
- [9]Kong S, Li C, Fang C, et al. Building a Speech Dataset and Recognition Model for the Minority Tu Language[J]. Applied Sciences, 2024, 14(15): 6795-6805.
- [10]Rahman A, Kabir M M, Mridha M F, et al. Arabic speech recognition: Advancement and challenges[J]. IEEE Access, 2024, 12: 39689-39716.
- [11]Eang C, Lee S. Predictive Maintenance and Fault Detection for Motor Drive Control Systems in Industrial Robots Using CNN-RNN-Based Observers[J]. Sensors, 2024, 25(1): 25-54.
- [12]Virupaksha A G, Nagel T, Lehmann F, et al. Modeling transient natural convection in heterogeneous porous media with Convolutional Neural Networks[J]. International Journal of Heat and Mass Transfer, 2024, 222: 125-149.

- [13]Ruan D, Wang J, Yan J, et al. CNN parameter design based on fault signal analysis and its application in bearing fault diagnosis[J]. Advanced Engineering Informatics, 2023, 55: 101877-101888.
- [14]LeCun Y, Boser B, Denker J, et al. Handwritten digit recognition with a back-propagation network[J]. Advances in Neural Information Processing Systems, 1989, 2: 396-404.
- [15]Krizhevsky A, Sutskever I, Hinton G E. Imagenet classification with deep convolutional neural networks[J]. Association for Computing Machinery, 2017, 60(6): 84-90.
- [16]LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.
- [17]徐彦威, 李军, 董元方, 等. YOLO 系列目标检测算法综述[J]. Journal of Frontiers of Computer Science & Technology, 2024, 18(9): 2221-2238.
- [18]梁振明, 黄影平, 宋卓恒, 等. 自动驾驶中基于深度学习的 3D 目标检测方法综述 [J]. 上海理工大学学报, 2024, 46(2): 103-119.
- [19]Xie T, Zhang W. Fast Intrusion Detection in High Voltage Zone of Electric Power Operations Based on YOLO and Homography Transformation Algorithm[C]. 2023 5th Asia Energy and Electrical Engineering Symposium (AEEES). IEEE, 2023: 686-691.
- [20]Hinton G E, Salakhutdinov R R. Reducing the dimensionality of data with neural networks[J]. Science, 2006, 313(5786): 504-507.
- [21]郭敏钢, 宫鹤. 基于 Tensorflow 对卷积神经网络的优化研究[J]. 计算机工程与应用, 2020, 56(1): 1906-0214.
- [22]Can F, Eyupoglu C. Fmsnet: A multi-stream cnn for multi-stereo image classification by feature map sharing[J]. IEEE Access, 2024, 12: 105566-105572.
- [23]文斌, 李知聪, 朱晗, 等. MHSACAE-CNN 在噪声下的电机轴承故障诊断[J]. 振动工程学报, 2023, 36(4): 1169-1178.
- [24]Zhang J, Wang R, Pei X, et al. A fast spiking neural network accelerator based on BP-STDP algorithm and weighted neuron model[J]. IEEE Transactions on Circuits and Systems II: Express Briefs, 2021, 69(4): 2271-2275.

- [25]Singh G, Agrawal S, Sohi B S. Handwritten Gurmukhi Digit Recognition System for Small Datasets[J]. *Traitement du Signal*, 2020, 37(4): 661-669.
- [26]He J, Li W, Xu J, et al. Research on Convolution Decomposition and Hardware Acceleration based on FPGA[C]. 2023 4th International Conference on Computer Engineering and Application (ICCEA). IEEE, 2023: 285-291.
- [27]Pitonak R, Mucha J, Dobis L, et al. Cloudsatnet-1: Fpga-based hardware-accelerated quantized cnn for satellite on-board cloud coverage classification[J]. *Remote Sensing*, 2022, 14(13): 3180-3192.
- [28]Lo C Y, Sham C W. Energy efficient fixed-point inference system of convolutional neural network[C]. 2020 IEEE 63rd International Midwest Symposium on Circuits and Systems (MWSCAS). IEEE, 2020: 403-406.
- [29]Yanamala R M R, Pullakandam M. A high-speed reusable quantized hardware accelerator design for CNN on constrained edge device[J]. *Design Automation for Embedded Systems*, 2023, 27(3): 165-189.
- [30]Chiranjeevi G N, Kulkarni S. Reconfigurable Platform Pre-Processing MAC Unit Design: For Image Processing Core Architecture in Restoration Applications[M]. *Latest Advances and New Visions of Ontology in Information Science*. IntechOpen, 2023.
- [31]Yan F, Zhang Z, Liu Y, et al. Design of Convolutional Neural Network Processor Based on FPGA Resource Multiplexing Architecture[J]. *Sensors*, 2022, 22(16): 59-67.
- [32]Xiao R, Shi J, Zhang C. FPGA implementation of CNN for handwritten digit recognition[C]. 2020 IEEE 4th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC). IEEE, 2020, 1: 1128-1133.
- [33]Wang A, Wang M, Jiang K, et al. A dual neural architecture combined SqueezeNet with OctConv for LiDAR data classification[J]. *Sensors*, 2019, 19(22): 4927-4941.
- [34]赵子龙, 赵毅强, 叶茂. 基于 FPGA 的多卷积神经网络任务实时切换方法[J]. *南京大学学报 (自然科学版)*, 2020, 56(2): 167-174.

- [35]Reed A L, Yang X, Sha S. Lightweight Neural Network Architectures for Resource-Limited Devices[C]. 2022 23rd International Symposium on Quality Electronic Design (ISQED). IEEE, 2022: 1-7.
- [36]Zhang J, Wang R, Pei X, et al. A fast spiking neural network accelerator based on BP-STDP algorithm and weighted neuron model[J]. IEEE Transactions on Circuits and Systems II: Express Briefs, 2021, 69(4): 2271-2275.
- [37]Jouppi N P, Yoon D H, Kurian G, et al. A domain-specific supercomputer for training deep neural networks[J]. Communications of the ACM, 2020, 63(7): 67-78.
- [38]朱郭益, 马胜, 张春元, 等. 异构多芯粒神经网络加速器[J]. 计算机辅助设计与图形学学报, 2023, 35(5): 811-818.
- [39]贾宏君, 周静, 乔开文, 等. 下一代互联网互联设备关键技术专利导航研究综述[J]. 中国科学: 信息科学, 2022, 52(5): 765-783.
- [40]Liang Y, Lu L, Jin Y, et al. An efficient hardware design for accelerating sparse CNNs with NAS-based models[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2021, 41(3): 597-613.
- [41]Shan D, Cong G, Lu W. A CNN Accelerator on FPGA with a Flexible Structure[C]. 2020 5th International Conference on Computational Intelligence and Applications (ICCIA). IEEE, 2020: 211-216.
- [42]Li T, He B, Zheng Y. Research and implementation of high computational power for training and inference of convolutional neural networks[J]. Applied Sciences, 2023, 13(2): 1003-1023.
- [43]Zheng Y, He B, Li T. Research on the Lightweight Deployment Method of Integration of Training and Inference in Artificial Intelligence[J]. Applied Sciences, 2022, 12(13): 6616-6632.
- [44]Wang Z, Li C, Lin P, et al. In situ training of feed-forward and recurrent convolutional memristor networks[J]. Nature Machine Intelligence, 2019, 1(9): 434-442.
- [45]Zenggang L I, Zhengyan W, Jingcheng S U N. Research and design of handwritten digital BP neural network based on FPGA[J]. Computer Engineering and Applications, 2020, 56(17): 251-257.

- [46]Li X, Li Z, Zhou L, et al. FOLD: Low-Level Image Enhancement for Low-Light Object Detection Based on FPGA MPSoC[J]. Electronics, 2024, 13(1): 230-243.
- [47]Nguyen D D, Nguyen D T, Le M T, et al. FPGA-SoC implementation of YOLOv4 for flying-object detection[J]. Journal of Real-Time Image Processing, 2024, 21(3): 63-81.
- [48]Reis M, Véstias M, Neto H. Designing Deep Learning Models on FPGA with Multiple Heterogeneous Engines[J]. ACM Transactions on Reconfigurable Technology and Systems, 2024, 17(1): 1-30.
- [49]Zhang H, Chen M, Ni M, et al. Automated feature map padding and transfer circuit for CNN inference[J]. IEICE Electronics Express, 2024, 21(22): 1-6.
- [50]Pan Z, Huang M, Zhu Q, et al. Developing a portable fluorescence imaging device for fish freshness detection[J]. Sensors, 2024, 24(5): 1401-1417.
- [51]Kim V H, Choi K K. A reconfigurable CNN-based accelerator design for fast and energy-efficient object detection system on mobile FPGA[J]. IEEE Access, 2023, 11: 59438-59445.
- [52]Liu W, Hu J, Qi J. Resistance Spot Welding Defect Detection Based on Visual Inspection: Improved Faster R-CNN Model[J]. Machines, 2025, 13(1): 33-49.
- [53]Han S, Mao H, Dally W J. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding[J]. Fiber, 2015, 56(4): 3-7.
- [54]Zhao H, Lai Z, Leung H, et al. Neural-network-based feature learning: Convolutional neural network[J]. Feature Learning and Understanding: Algorithms and Applications, 2020: 219-251.
- [55]Wei H, Wang Z, Hua G. Dynamically mixed group convolution to lighten convolution operation[C]. 2021 4th International Conference on Artificial Intelligence and Big Data (ICAIBD). IEEE, 2021: 203-206.
- [56]Nanni L, Lumini A, Ghidoni S, et al. Stochastic activation function layers for convolutional neural networks[J]. Preprints, 2020, 34(3):1478-1488 .
- [57]Zhang G, Kong Y, Li W, et al. Lightweight deep learning model for logistics parcel detection[J]. The Visual Computer, 2024, 40(4): 2751-2759.

- [58]Wu J, Zhang L. Deep Model Pruning By Parallel Pooling Attention Block[C]. 2022 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT). IEEE, 2022: 847-851.
- [59]Lee H, Yoo J S, Jung S W. RefQSR: Reference-Based Quantization for Image Super-Resolution Networks[J]. IEEE Transactions on Image Processing. 2024, 33: 2823-2834.
- [60]Mirzadeh S I, Farajtabar M, Li A, et al. Improved knowledge distillation via teacher assistant[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2020, 34(04): 5191-5198.
- [61]Shan D, Cong G, Lu W. A CNN Accelerator on FPGA with a Flexible Structure[C]. 2020 5th International Conference on Computational Intelligence and Applications (ICCIA). IEEE, 2020: 211-216.
- [62]Youssef E, Elsemery H A, El-Moursy M A, et al. Energy adaptive convolution neural network using dynamic partial reconfiguration[C]. 2020 IEEE 63rd International Midwest Symposium on Circuits and Systems (MWSCAS). IEEE, 2020: 325-328.
- [63]李慧. 手写体数字识别的卷积神经网络研究与 FPGA 实现[D]. 西安: 西安石油大学, 2021.
- [64]Mujawar S, Kiran D, Ramasangu H. An efficient CNN architecture for image classification on FPGA accelerator[C]. 2018 Second International Conference on Advances in Electronics, Computers and Communications (ICAIECC). IEEE, 2018: 1-4.
- [65]Nagarajan M, Muthaiah R, Teekaraman Y, et al. Power and area efficient cascaded effectless GDI approximate adder for accelerating multimedia applications using deep learning model[J]. Computational Intelligence and Neuroscience, 2022, 2022(1): 1-15.
- [66]Yanamala R M R, Pullakandam M. A high-speed reusable quantized hardware accelerator design for CNN on constrained edge device[J]. Design Automation for Embedded Systems, 2023, 27(3): 165-189.

- [67]Cho M, Kim Y. FPGA-based convolutional neural network accelerator with resource-optimized approximate multiply-accumulate unit[J]. Electronics, 2021, 10(22): 1-6.
- [68]Feng G, Hu Z, Chen S, et al. Energy-efficient and high-throughput FPGA-based accelerator for Convolutional Neural Networks[C]. 2016 13th IEEE International Conference on Solid-State and Integrated Circuit Technology (ICSICT). IEEE, 2016: 624-626.
- [69]Cheng J. Design and implementation of convolutional neural network accelerator based on fpga[J]. Frontiers in Computing and Intelligent Systems, 2023, 3(1): 158-161.
- [70]Zhen-yu Y I N, Guang-yuan X U, Fei-qing Z, et al. Design and implementation of convolution neural network unit based on Zynq platform[J]. Journal of Chinese Computer Systems, 2022, 43(2): 231-235.
- [71]许宏达. 基于 Zynq 平台的目标检测算法研究与实现[D]. 成都: 电子科技大学, 2021.
- [72]严敏佳. CNN 目标检测系统在嵌入式平台的设计与实现[D]. 成都: 电子科技大学, 2019.
- [73]党涵. 目标检测算法在 Zynq-7000SoC 上的设计与实现[D]. 成都: 电子科技大学, 2024.